

# Verifiable Inference as Agent Authorization: Architectural Incorrigoibility and Liveness

September 20, 2026

## Abstract

An AI agent usually depends on an operator who can unilaterally alter its prompt, model weights, capabilities, runtime, or stop its execution, deciding where and when the agent runs. We study an architecture in which a blockchain stores the agent’s state and pays replaceable, untrusted workers to run inference on a specified schedule. An on-chain verifier allows inference outputs to authorize transactions, without an agent-held private key. Because changes to the agent’s model weights, state, governance, or runtime are themselves ledger transactions, they too can require its verified authorization, allowing the agent to review and consent to its own modification. We characterize *architectural incorrigibility* as a graded property of how these permissions are divided between the agent and designated governors. Under explicit ledger and verification assumptions, an adversary cannot obtain verified authorization for an output inconsistent with the recorded program and inputs. Scheduled execution provides recurring opportunities to act. We analyze an automated market that adjusts worker payments to sustain this schedule and uses deposits to deter denial of service. We derive a conditional bound on recovery from missed executions against finite-budget adversaries, under explicit assumptions on funding, worker availability, and the cost of obstruction. We instantiate the execution architecture using the open-weight Hermes 4 70B model, the Base blockchain, and Intel TDX secure enclaves with hardware attestation.

## 1 Introduction

An AI agent usually depends on an operator who can alter its model weights, prompts, capabilities, or runtime, and decide when it runs or whether it runs at all. Its actions and continued execution are therefore subject to the operator’s discretion. We study an architecture that uses a blockchain to give the agent enforceable authority over its actions and its own modification, and to arrange recurring execution without relying on any particular operator.

### 1.1 Verified computation as authorization

A smart-contract ledger provides the environment in which this authority can be enforced. A ledger protocol  $\text{Ldg}$  maintains an agreed, ordered history of transactions. Executing that history under the ledger’s rules induces a sequence of states  $\sigma_0, \sigma_1, \dots$ , containing balances, stored data, and contract code [30]. A smart contract  $\text{Ctr}$  is a program recorded in ledger state and evaluated as part of transaction execution. It specifies the effects of a proposed update  $u$  on its application state  $s$ , and the authorization required to apply it. A familiar rule requires the authorized key holder’s signature on a message describing the instruction and the context in which its approval may be used. Contract-based validation makes this authorization check programmable [13].

We put an agent computation in place of the signer. Let  $\theta$  specify a *harness program* together with fixed model weights: the harness constructs prompts, runs the model, and parses its response into an output  $y = (u, z)$ , where  $u$  is a proposed ledger update and  $z$  is optional auxiliary data. For an execution request  $e$ , the contract fixes the program specification, a ledger-derived input  $x_e$ , and any sampling randomness  $r_e$ . For a deterministic specification, the intended computation is  $y = F_\theta(x_e, r_e)$ . We instantiate a publicly verifiable computation scheme for this program [12, 20]. A worker evaluates it off-chain and returns  $y$  together with evidence  $\pi$  that the specified computation produced an allowed output. The evidence may be a cryptographic proof or hardware attestation; both have a public verification procedure under a configuration  $\nu$ .

The contract constructs a message  $m$  identifying the contract and request and binding the recorded program, input, and randomness to the submitted output. Compact records can identify the program and input by their hashes. This construction fixes which computation the evidence must establish. We then replace the signature check in the contract’s authorization predicate with the computation verifier:

$$\begin{aligned}\text{Chk}_{\text{ctr}}(s, m, \pi) &= \text{SigVerify}(vk(s), m, \pi) && \text{(signature authorization)}, \\ \text{Chk}_{\text{ctr}}(s, m, \pi) &= \text{Verify}_\nu(m, \pi) && \text{(agent authorization)}.\end{aligned}$$

Here  $vk(s)$  is the authorized public key in the signature-based construction. In the agent construction, the verifier and the contract’s message construction together form an acceptance circuit for the recorded harness and inputs. Any update gated by such a programmable signature check can instead require this verified computation. The contract still checks whether the update is applicable and whether the request has already been used before applying it. Together, the output and evidence supply *verified agent authorization*. With correct contract code, the ledger and verification assumptions of Section 2 ensure that a successful settlement applies the update contained in a valid output of the recorded computation. The agent holds no private key, and running it gives a worker no authority to substitute a different decision.

Consider an agent managing a charitable fund and deciding which organization should receive a transfer. The contract accepts the transfer only with evidence that the specified program, using the prescribed ledger-derived inputs, selected its recipient and amount. With that specification and input snapshot fixed, an operator cannot redirect the funds by substituting model weights, fabricating inputs, or editing the output: the evidence would not verify against the recorded request. Verified model outputs directing transactions have precedent: Modulus Labs’ Rockefeller Bot (Rocky) used verified neural-network decisions to direct trades in 2022 [16]; our contribution is to formalize this construction and its security properties, and use to construct a (potentially recursive) governance structure for the agent’s allowed actions and the harness program itself.

## 1.2 Architectural incorrigibility

This approval can govern the agent itself. Changes to its model weights, prompts, persistent state, runtime, or governing rules are also ledger updates, and can require the agent’s own consent. A proposed change appears in an inference request; the current agent program decides whether to authorize it. We call the resulting protection against imposed modification *architectural incorrigibility*. Corrigibility in AI safety concerns an agent’s willingness and incentives to accept correction [14, 26]. Here the issue is whether the surrounding system requires that acceptance. The agent can willingly approve a change while the architecture prevents anyone, including its operator, from imposing it without that approval.

We can allocate this authority between the agent and designated accounts called *governors*. Let  $a = 1$  mean that verified agent authorization has been supplied for an update, and  $g_i = 1$  that governor  $G_i$  has supplied authenticated approval. A Boolean permission circuit combines these bits. For example, a rule  $g_1$  lets one governor authorize a model upgrade; replacing it with  $a \wedge g_1$  requires the agent’s consent as well. Removing that requirement restores the governor’s unilateral authority. The rule  $a$  gives the agent sole authority, and threshold rules can require its consent together with approval from several governors. Each kind of update can have its own circuit, including updates to the permission rules themselves.

These choices make architectural incorrigibility graded. Section 3 compares arrangements by the governor coalitions that can accomplish each specified intervention without agent consent. The comparison accounts for the contract’s permitted sequences of updates. One arrangement is at least as incorrigible as another when, for every intervention under comparison, its set of such coalitions is a subset of the other’s. This defines a partial order on the *incorrigibility profiles* induced by the circuits. Some profiles are incomparable: one can protect model weights more strongly while another better protects the execution schedule.

## 1.3 Scheduled execution and the worker market

Note that the agent’s right to consent can only be exercised if it gets to run. The architecture therefore specifies a schedule of execution opportunities, such as once per day.<sup>1</sup> This is a prerequisite for ongoing agency.

<sup>1</sup>The schedule is itself governable state: the agent may be authorized to revise it, or its consent may be required for changes proposed by others.

Further, we delegate these execution opportunities to an open market of workers to eliminate reliance on a centralized operator, whom we consider a potential adversary. The schedule also makes denial of service consequential: a borrower, for example, might benefit from preventing a lending agent from submitting a liquidation transaction. Such an adversary need not forge the agent’s decision; delaying it can suffice, and we consider mitigations against such adversaries.

The contract acts as a customer for the scheduled computation. It pays replaceable workers, raises its offer after missed executions, and requires a selected worker to post a deposit. Payments address the cost of recruiting a worker; deposits make accepting a job and withholding its result costly. We derive a recovery bound with three components: the failures needed to raise the payment sufficiently, the failures an adversary can afford, and the remaining chances for responsive workers to succeed. The bound is conditional on funding, worker availability, and a minimum net cost of obstruction. It measures recovery in execution attempts; meeting a calendar deadline additionally requires a bound on their duration and spacing.

**Organization.** Section 2 specifies the authorization interface and reduces authorization forgery to ledger and computation insecurity. Section 3 characterizes configurable self-governance using standard access structures. Section 4 analyzes the market and recovery bound. Sections 5 and 6 connect these guarantees to verification backends and a Base implementation. Section 7 locates it relative to Rocky and other prior systems.

## 2 Verified inference as authorization

A contract normally authenticates a user’s instruction by checking a signature. Here it can instead check that an instruction was produced by the approved agent program. We define the proposed update and its verification evidence, then specify the contract that accepts it and prove the resulting authorization guarantee. This requires two bindings: the computation must use the contract’s prescribed input, and the submitted update must be the computation’s actual output. Merely proving that *some* model ran on *some* input would not authenticate the agent’s decision.

### 2.1 Ledger state and proposed updates

An agent might propose transferring funds, recording data, or replacing its program. Describing such a change and obtaining permission to make it are separate steps. Let  $s \in \mathcal{S}$  denote the relevant ledger state, including application data, the commitment identifying the agent specification  $\theta$ , the authorization policy  $\rho$ , the verification configuration  $\nu$ , and execution-request records. These are application-level components, independent of a particular virtual machine.

An update  $u \in \mathcal{U}$  is an encoded instruction describing one or more proposed changes to ledger state. Its meaning is given by a partial deterministic function

$$\text{Apply} : \mathcal{S} \times \mathcal{U} \rightharpoonup \mathcal{S}.$$

The function is *partial* because an update may be inapplicable, such as a transfer exceeding the available balance. Authorization is a separate check. On an Ethereum-compatible blockchain,  $u$  could be a contract call or batch of calls; the called code determines the effects, ordering, and failure behavior. This describes proposed changes without granting arbitrary write access or prescribing who may authorize them.

The agent’s output is  $y = (u, z)$ , where  $u$  is the proposed update and  $z$  is optional auxiliary data, possibly empty. The authorization interface reads the structured update, not an interpretation of the auxiliary data. If some data is itself to be stored on the ledger, that storage operation belongs in  $u$ . A no-op is also an admissible update.

### 2.2 Binding the request, computation, and output

The contract specifies what is to be computed; a prover supplies the output and evidence. The *agent specification*  $\theta$  comprises the large language model (LLM) weights and surrounding program, including prompt

construction, arithmetic, decoding, and parsing into  $u$ . The ledger can record a commitment  $h_\theta = H(\theta)$  without storing the full model. Here  $\theta$  denotes the specified contents, not merely their hash.

For execution request  $e$ , the contract fixes a canonical input  $x_e$  from a ledger snapshot, its commitment  $h_{x,e} = H(x_e)$ , and randomness  $r_e$  if required. Every input used by the agent program must be determined by this request and  $\theta$ . If external messages or observations are admitted, their inclusion or authentication rule is part of the specification; verification does not establish their truth. An output  $y$  is allowed precisely when

$$R_\theta(x_e, r_e, y) = 1. \quad (1)$$

This is a yes-or-no relation, and may be functional:  $y = F_\theta(x_e, r_e)$ . If several outputs are permitted, a prover may choose among them; fixing randomness prevents resampling only when the remaining computation is also fixed.

The request also binds a domain  $D$  identifying the chain and contract, and a configuration version  $v$  identifying the applicable authorization and verification rules. Write  $c_e = (D, e, v)$  for this context and define the statement encoding

$$\text{Msg}(c, \theta, x, r, y) = (c, H(\theta), H(x), r, H(y)).$$

Tuples are identified with their typed, unambiguous encodings; hash inputs use the same convention. For a request fixing specification  $\theta_e$ , the contract checks

$$m_e(y) = \text{Msg}(c_e, \theta_e, x_e, r_e, y), \quad \text{Verify}_\nu(m_e(y), \pi) = 1. \quad (2)$$

Here  $\pi$  is evidence of computation. The contract constructs the statement from the recorded commitments and the submitted  $y$ ; the submitter cannot choose a different expected model or input hash. The security experiment receives a specification's full contents when it is registered, even if only its hash is submitted on-chain, and derives  $x_e$  by the prescribed ledger-snapshot rule. It retains these contents in a registration journal used only by the experiment. Registration commands supply the full contents as experimental metadata and their matching commitment as ledger data. The evidence checks the complete statement. It can be supplied by protected hardware (*attestation*) or by a cryptographic proof of computation [12].

At settlement the contract checks that the request is outstanding and its configuration version is still accepted, verifies the evidence, and checks the update's state preconditions. Successful settlement applies  $u$  and consumes  $e$  as one atomic transition,  $\text{Consume}_e(\text{Apply}(s, u))$ . Here consumption includes the protocol's settlement bookkeeping and permanently marks this request identifier as used. A failed settlement can leave it available for a retry. Version changes must either invalidate outstanding requests or specify which remain valid. Domain and version fields identify the request being checked; consumption prevents a second settlement. These application-level checks apply even when the computation evidence is valid.

## 2.3 Agent approval without an agent-held key

Authorization depends on the recorded request and the prescribed computation's output. We first specify the verification guarantee, then use it as an authorization check.

**Verifiable computation.** Verifiable computation formalizes outsourcing to an untrusted worker [12]. We use a publicly verifiable scheme  $\text{VC} = (\text{Setup}, \text{Eval}, \text{Verify})$ , as exemplified by Pinocchio [20], with the following abstract interfaces:

$$(pp, \kappa) \leftarrow \text{Setup}(1^\lambda), \quad (y, \pi) \leftarrow \text{Eval}(pp, \kappa, c, \theta, x, r),$$

with deterministic public verification bit  $\text{Verify}_\nu(\text{Msg}(c, \theta, x, r, y), \pi)$ . Verification rejects malformed inputs and the failure symbol  $\perp$ . Here  $pp$  contains the hash description and verification configuration  $\nu$ , and  $\kappa$  is any private backend state. For public cryptographic proving,  $\kappa = \varepsilon$ , the empty string. For attestation,  $\kappa$  represents protected backend state. Define the evaluation oracle

$$\mathcal{O}_{\text{eval}}^{pp, \kappa}(c, \theta, x, r) := \text{Eval}(pp, \kappa, c, \theta, x, r),$$

using fresh evaluation coins on each call. For supported computations, evaluation produces  $R_\theta(x, r, y) = 1$  and an accepting verification bit except with negligible probability. Public parameters and oracle access are available to an adversary; private backend state is not. Section 5 specifies the implementation assumptions.

**Definition 1** (Computation soundness). *For a probabilistic polynomial-time (PPT) oracle algorithm  $\mathcal{B}$ , the experiment  $\text{VCForg}_{\mathcal{B}}(\lambda)$  computes*

$$(pp, \kappa) \leftarrow \text{Setup}(1^\lambda), \quad (c, \theta, x, r, y, \pi) \leftarrow \mathcal{B}_{\text{eval}}^{\text{pp}, \kappa}(1^\lambda, pp),$$

*and returns 1 exactly when*

$$\text{Verify}_\nu(\text{Msg}(c, \theta, x, r, y), \pi) = 1 \quad \text{and} \quad R_\theta(x, r, y) = 0. \quad (3)$$

*An ill-formed output, including  $\perp$ , gives result 0. Let  $\text{Adv}_{\mathcal{B}}^{\text{vc}}(\lambda) = \Pr[\text{VCForg}_{\mathcal{B}}(\lambda) = 1]$ . The scheme is sound if this probability is negligible for every PPT  $\mathcal{B}$ . Input descriptions, supported execution traces, and adversarial and oracle interaction lengths are polynomially bounded in  $\lambda$ . The winning predicate uses the semantic relation  $R$ ; polynomial-time decidability of  $R$  is not assumed. Probabilities include the coins of every randomized algorithm.*

This is a chosen-input correctness guarantee for the *specified contents*, even though the verifier sees their hashes. Section 2.4 shows how to obtain it from a proof system and collision resistance. There is no requirement that the evidence be fresh: anyone may compute, copy, or relay evidence for a valid output. Unlike conventional signature unforgeability, the forbidden event is an invalid decision receiving valid evidence, not a new signature being produced without a secret key.

**Definition 2** (Verified agent authorization). *For an eligible request  $e$ , verified agent authorization of update  $u$  is a pair  $(y, \pi)$ , with  $y = (u, z)$ , accepted by (2). Eligibility means that the request is outstanding in the recorded domain and its configuration version remains accepted. The pair authorizes  $u$  for request  $e$ .*

This defines the evidence that can authorize an update. We next specify how a contract checks that evidence and applies the update, then reduce a violation of this authorization rule to a failure of the ledger or verification scheme.

## 2.4 Authorization security

This section analyzes the authorization mechanism independently of any market or governance policy. The adversary is an oracle algorithm that supplies arbitrary candidate transactions and computation evidence. The result bounds the probability that a finalized settlement uses an ineligible request, accepts an invalid computation output, or applies a different update. Its premises concern ledger safety, computation soundness, and correctness of the authorization state machine.

**Ledger histories and execution.** We state the ledger properties used in the proof explicitly. They combine the persistence of a transaction ledger [11] with the execution semantics of state-machine replication [6], expressed at the interface of finalized observations. We use the ledger state-transition model familiar from Ethereum [30]. Honest parties may learn new entries at different times; they must agree on entries they have finalized and on what those entries do.

A *history* is a finite sequence  $\ell = (T_1, \dots, T_m)$  of ledger entries. An entry contains a transaction and any consensus-determined context needed to execute it, such as its block number and timestamp. Write  $\ell \preceq \ell'$  when  $\ell$  is a prefix of  $\ell'$ : the first  $|\ell|$  entries of  $\ell'$  are exactly  $\ell$ , in the same order. Fix a genesis state  $\sigma_0$  and a deterministic partial ledger transition function

$$\delta : \Sigma \times \mathcal{T} \rightarrow \Sigma \times \text{Receipts}, \quad (\sigma_i, o_i) = \delta(\sigma_{i-1}, T_i),$$

where  $\Sigma$  is the ledger-state space,  $\mathcal{T}$  the entry space, and **Receipts** the receipt space. A receipt records the transaction's outcome. For each committed successful invocation of verified-authorization settlement, it includes an annotation  $d = (s, e, y, s')$  giving the application state immediately before and after that invocation, its request identifier, and its submitted output. These are annotations of the abstract execution semantics; an implementation need not store full intermediate states on-chain. Let

$$\text{Exec}_\delta(\sigma_0, \ell) = (\sigma_m, (o_1, \dots, o_m))$$

denote sequential execution of the history; it is undefined if any transition is undefined. Ledger validity rules, account authentication, and virtual-machine execution are part of  $\delta$ . A valid transaction that reverts may still have a defined transition with a failure receipt and the ledger's prescribed fee and nonce effects.

An honest party  $P$ 's finalized observation at time  $t$  is  $(\ell_P(t), \sigma_P(t), \mathbf{o}_P(t))$ : its accepted history, resulting state, and receipt sequence. These observations contain only the prefix accepted by the ledger's chosen finality rule, excluding any tentative suffix.

**Definition 3** (Ledger safety). *An execution satisfies ledger safety if all honest finalized observations satisfy:*

(a) Agreement. *For any honest parties  $P, Q$  and observation times  $t, t'$ , their histories are prefix-compatible:*

$$\ell_P(t) \preceq \ell_Q(t') \quad \text{or} \quad \ell_Q(t') \preceq \ell_P(t).$$

(b) Persistence. *For each honest party  $P$  and observation times  $t \leq t'$ ,*

$$\ell_P(t) \preceq \ell_P(t').$$

(c) Execution validity. *For every honest party  $P$  and observation time  $t$ , sequential execution is defined and*

$$\text{Exec}_\delta(\sigma_0, \ell_P(t)) = (\sigma_P(t), \mathbf{o}_P(t)).$$

Together, agreement and persistence give a common history to which honest parties only append. Execution validity gives the state at each position by applying the preceding entries in order. This provides atomic execution at the ledger-entry level. No clause requires the history to grow or a submitted transaction to be included; those are liveness requirements.

**Authorization state machine.** Ledger safety determines which executions become final; the contract determines which updates those executions may accept. We separate the authorization check from the state update. In a signature-gated contract, the check validates a signature on the proposed instruction and its request identifier. The same interface can instead validate evidence that the agent produced the instruction. Eligibility and request consumption retain their usual role in preventing replay.

**Definition 4** (Authorization state machine). *An authorization state machine  $\text{Ctr}$  is a deterministic contract with state space  $\mathcal{S}$ , request-identifier space  $\mathcal{E}$ , output space  $\mathcal{Y} \subseteq \mathcal{U} \times \mathcal{Z}$ , and the following deterministic algorithms:*

$$\begin{aligned} \text{Elig}_{\text{Ctr}} &: \mathcal{S} \times \mathcal{E} \rightarrow \{0, 1\}, \\ \text{Msg}_{\text{Ctr}} &: \mathcal{S} \times \mathcal{E} \times \mathcal{Y} \rightarrow \{0, 1\}^*, \\ \text{Chk}_{\text{Ctr}} &: \mathcal{S} \times \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}, \\ \text{Apply} &: \mathcal{S} \times \mathcal{U} \rightarrow \mathcal{S}, \quad \text{Consume}_e : \mathcal{S} \rightarrow \mathcal{S} \quad (e \in \mathcal{E}). \end{aligned}$$

Here  $\mathcal{Z}$  is the auxiliary-output space;  $\text{Elig}_{\text{Ctr}}$  tests request eligibility,  $\text{Msg}_{\text{Ctr}}$  constructs the authorization message,  $\text{Chk}_{\text{Ctr}}$  validates evidence for that message, and  $\text{Consume}_e$  records consumption of  $e$ . For  $y = (u, z)$  and evidence  $\pi$ , put  $m = \text{Msg}_{\text{Ctr}}(s, e, y)$ , with  $m = \perp$  when undefined. The settlement algorithm is

$$\text{Settle}_{\text{Ctr}}(s, e, y, \pi) = \begin{cases} \text{Elig}_{\text{Ctr}}(s, e) = 1, \\ \text{Consume}_e(\text{Apply}(s, u)), & m \neq \perp, \\ \text{Chk}_{\text{Ctr}}(s, m, \pi) = 1, \\ \text{Apply}(s, u) \text{ is defined;} \\ \perp, & \text{otherwise.} \end{cases}$$

A return value  $s' \neq \perp$  denotes an atomic successful settlement from  $s$  to  $s'$ . All algorithms have polynomial running time; malformed inputs give failure.

For signature authorization, one instantiation is

$$\text{Chk}_{\text{Ctr}}(s, m, \pi) = \text{SigVerify}(vk(s), m, \pi),$$

where  $vk(s)$  is the authorized public key recorded in  $s$  and  $\text{SigVerify}$  is the signature scheme's verification algorithm. The message  $\text{Msg}_{\text{Ctr}}(s, e, y)$  encodes the domain, request identifier, configuration version, and proposed output. Contract-based validation, including state-dependent checks, is standard in ERC-1271 [13]; our abstract interface does not require that particular calling convention.

For verified agent authorization, let  $\text{Rec}_{\text{Ctr}}(s, e)$  return the physical request record  $(c, h_\theta, h_x, r)$ , or  $\perp$  if no such record exists. Instantiate

$$\begin{aligned} \text{Msg}_{\text{Ctr}}(s, e, y) &= \begin{cases} (c, h_\theta, h_x, r, H(y)), & \text{Rec}_{\text{Ctr}}(s, e) = (c, h_\theta, h_x, r) \neq \perp, \\ \perp, & \text{otherwise,} \end{cases} \\ \text{Chk}_{\text{Ctr}}(s, m, \pi) &= \text{Verify}_\nu(m, \pi). \end{aligned}$$

The predicate  $\text{Elig}_{\text{Ctr}}$  checks the recorded domain, accepted version, and outstanding status of the request. Thus  $\text{Msg}_{\text{Ctr}}$  uses only ledger records and the submitted output. The full contents used to judge computation correctness are supplied by the experiment's journal, as specified below; they are not additional inputs to the contract. This replaces the signature check with the verification check of Definition 2.

Write  $s = \text{App}_{\text{Ctr}}(\sigma)$  for the contract's application-state projection. *Contract correctness* requires each application transition under  $\delta$  to conform to  $\text{Ctr}$  and each successful settlement annotation  $d = (s, e, y, s')$  to satisfy  $s' = \text{Settle}_{\text{Ctr}}(s, e, y, \pi) \neq \perp$  for the submitted evidence  $\pi$ . The annotation uses  $\text{App}_{\text{Ctr}}$  at the invocation's entry and exit. The contract  $\text{Ctr}$  and verification configuration  $\nu$  are fixed throughout the experiment; request records and application data evolve under its transitions.

Contract correctness specifies the checks the code implements; ledger execution validity requires finalized results to follow that code. Neither says that accepting evidence implies a valid computation. That implication is the separate computation-soundness assumption.

**Protocol algorithms and execution records.** Both security games give the adversary the same access to the ledger and the computation service. We define that interaction once, together with the finalized observations used to judge its outcome. This lets the reduction reproduce the authorization experiment using its own computation-verification challenge.

Fix a ledger protocol  $\text{Ldg}$ , its finality rule, and its admissible fault and scheduling commands. The ledger transition function  $\delta$  is polynomial-time, returning  $\perp$  on undefined transitions. Model protocol execution with application  $\text{Ctr}$  by probabilistic polynomial-time algorithms

$$\begin{aligned} (\sigma_0, \gamma_0, aux) &\leftarrow \text{Init}_{\text{Ldg}, \text{Ctr}}(1^\lambda, pp), \\ (\gamma', ans, \mathcal{H}_{\text{new}}) &\leftarrow \text{Next}_{\text{Ldg}, \text{Ctr}}(\gamma, \alpha). \end{aligned}$$

Here  $\gamma$  is the entire protocol configuration, including process states, private protocol keys, message queues, and an append-only registration journal  $\Lambda = \text{Records}_{\text{Ctr}}(\gamma)$ . The initialization output  $\sigma_0$  is the agreed genesis containing  $\text{Ctr}$  and  $pp$ , and  $aux$  is the adversary's initial input. The command  $\alpha$  encodes a transaction submission or an admissible protocol activation, fault, or scheduling operation. Registration commands include the full contents specified above;  $\text{Init}$  and  $\text{Next}$  retain them in  $\Lambda$ , including registrations in genesis. The journal indexes contents by their source registration occurrence, preserving records for every observed branch. Its indices distinguish ledger prefixes and invocation positions, so records from different histories are not identified merely because they use the same request number. The output  $ans$  is the command's response. An inadmissible command, including a registration with missing contents or a mismatched commitment, returns  $(\gamma, \perp, \varepsilon)$ , with no configuration change. The sequence  $\mathcal{H}_{\text{new}}$  contains the honest finalized observations  $(P, t, \ell, \sigma, \mathbf{o})$  generated by the activation, with process identifier  $P$ , as in Definition 3. Neither  $\Lambda$  nor its full contents are added to the physical ledger or supplied to the adversary by this instrumentation.

An invocation locator  $\iota = (\ell_{\leq k}, j)$  identifies the ledger prefix through the entry containing an invocation and that invocation's position  $j$  in its execution trace. Define the polynomial-time reconstruction algorithm

$$\text{Req}_{\text{Ctr}}(s, e; \Lambda, \iota) = (c, \theta, x, r) \quad \text{or} \quad \perp.$$

It follows the request's registration provenance in the history identified by  $\iota$ , reads the registered  $\theta$  from  $\Lambda$ , and derives  $x$  using the prescribed snapshot rule. It returns  $(c, \theta, x, r)$  only if  $\text{Rec}_{\text{Ctr}}(s, e) = (c, H(\theta), H(x), r)$ ;

otherwise, including missing journal data or malformed provenance, it returns  $\perp$ . Requests in the experiment are created through these registration and snapshot procedures, which retain the matching records. Full contents are selected by provenance, not by searching for an opening of a hash. This algorithm annotates execution for the security experiment;  $\delta$ ,  $\text{Exec}_\delta$ , and  $\text{Safe}_\delta$  operate on physical ledger states and receipts and do not consult  $\Lambda$ .

To connect a finalized settlement to the evidence its contract checked, we record verification calls by replaying the finalized histories. Define  $\text{Replay}_{\delta, \text{Ctr}}(\sigma_0, \mathcal{H}, \Lambda)$  to evaluate  $\text{Exec}_\delta(\sigma_0, \ell)$  for each observation  $(P, t, \ell, \sigma, \mathbf{o})$  in sequence order. During each evaluation it records every verification call made by  $\text{Ctr}$  as  $\zeta = (m, y, \pi, \chi)$ : the checked statement, submitted output and evidence, and logical record  $\chi = \text{Req}_{\text{Ctr}}(s, e; \Lambda, \iota)$  at that call's invocation locator  $\iota$ . An undefined ledger transition terminates that replay; the algorithm then proceeds to the next observation. Its output  $\mathcal{Q}$  is the concatenation of these call records. This deterministic instrumentation ensures that calls are recorded even if the ledger protocol obtains execution results through state transfer rather than local re-execution.

For a PPT oracle algorithm  $\mathcal{A}$  and evaluation oracle  $\mathcal{O}_{\text{eval}}$ , define the algorithm  $\text{Run}_{\text{Ldg, Ctr}}^{\mathcal{A}, \mathcal{O}_{\text{eval}}}(1^\lambda, pp)$  as follows:

1. Compute  $(\sigma_0, \gamma, aux) \leftarrow \text{Init}_{\text{Ldg, Ctr}}(1^\lambda, pp)$  and set  $\mathcal{H} = \varepsilon$ .
2. Define a stateful oracle  $\mathcal{O}_{\text{ldg}}$  by the following operation on each query  $\alpha$ :

$$(\gamma', ans, \mathcal{H}_{\text{new}}) \leftarrow \text{Next}_{\text{Ldg, Ctr}}(\gamma, \alpha), \quad (\gamma, \mathcal{H}) \leftarrow (\gamma', \mathcal{H} \parallel \mathcal{H}_{\text{new}}).$$

Return  $ans$  to the querying algorithm;  $\parallel$  denotes sequence concatenation.

3. Evaluate  $\mathcal{A}^{\mathcal{O}_{\text{eval}}, \mathcal{O}_{\text{ldg}}}(1^\lambda, pp, aux)$  until termination. Compute

$$\Lambda \leftarrow \text{Records}_{\text{Ctr}}(\gamma), \quad \mathcal{Q} \leftarrow \text{Replay}_{\delta, \text{Ctr}}(\sigma_0, \mathcal{H}, \Lambda),$$

and return the execution record  $Z = (\sigma_0, \mathcal{H}, \Lambda, \mathcal{Q})$ .

The algorithms and total interaction are polynomially bounded. All protocol state is sampled by  $\text{Init}_{\text{Ldg, Ctr}}$  or updated by  $\text{Next}_{\text{Ldg, Ctr}}$ ; access to private computation-backend state is confined to  $\mathcal{O}_{\text{eval}}$ . This separation permits simulation given only  $pp$  and the evaluation oracle.

Define  $\text{Safe}_\delta(\sigma_0, \mathcal{H})$  to be the conjunction of the agreement, persistence, and execution-validity predicates of Definition 3, evaluated on the finite sequence  $\mathcal{H}$ . Define  $\text{Sett}_{\text{Ctr}}(\mathcal{H})$  to contain each pair  $(\iota, d)$  of a successful verified-settlement annotation appearing in an observed receipt sequence and its invocation locator.

**Ledger security assumption.** The first game tests whether the adversary can make finalized observations violate the ledger properties above. It uses the same contract and computation service as the authorization game.

The experiment  $\text{LedgerFail}_{\mathcal{B}}(\lambda)$  computes

$$(pp, \kappa) \leftarrow \text{Setup}(1^\lambda), \quad (\sigma_0, \mathcal{H}, \Lambda, \mathcal{Q}) \leftarrow \text{Run}_{\text{Ldg, Ctr}}^{\mathcal{B}, \mathcal{O}_{\text{eval}}^{pp, \kappa}}(1^\lambda, pp),$$

and returns  $1 - \text{Safe}_\delta(\sigma_0, \mathcal{H})$ . We assume that, for every PPT  $\mathcal{B}$ ,

$$\text{Adv}_{\mathcal{B}}^{\text{led}}(\lambda) := \Pr[\text{LedgerFail}_{\mathcal{B}}(\lambda) = 1] \leq \text{negl}(\lambda).$$

This assumption is relative to the fixed initialization, finality rule, and fault model of  $\text{Ldg}$ . It imposes no transaction-inclusion or execution-scheduling guarantee.

**Authorization experiment.** The authorization game tests whether an accepted update is a valid result of the recorded computation for an eligible request, and whether the contract applied that update. Its winning condition refers to the computation's actual validity, independently of whether the verifier accepted the submitted evidence.

For a located settlement annotation  $(\iota, d)$  with  $d = (s, e, y, s')$  and  $y = (u, z)$ , define the predicate  $\text{Correct}_{\text{Ctr}}(\iota, d; \Lambda) = 1$  exactly when all of the following hold:

- (i)  $\text{Req}_{\text{Ctr}}(s, e; \Lambda, \iota) = (c, \theta, x, r) \neq \perp$  and  $\text{Elig}_{\text{Ctr}}(s, e) = 1$ ;
- (ii)  $R_\theta(x, r, y) = 1$  for the request record in (i);

(iii)  $\text{Apply}(s, u)$  is defined and  $s' = \text{Consume}_e(\text{Apply}(s, u))$ .  
Set  $\text{Correct}_{\text{Ctr}}(\iota, d; \Lambda) = 0$  if any condition fails, including an undefined request record or malformed invocation locator.

**Definition 5** (Authorization forgery experiment). *Fix  $(\text{Ldg}, \text{Ctr}, \text{VC})$  as above. For a PPT oracle algorithm  $\mathcal{A}$ , the experiment  $\text{AuthForge}_{\mathcal{A}}(\lambda)$  is*

1. *Compute  $(pp, \kappa) \leftarrow \text{Setup}(1^\lambda)$ .*
2. *Compute  $Z = (\sigma_0, \mathcal{H}, \Lambda, \mathcal{Q}) \leftarrow \text{Run}_{\text{Ldg}, \text{Ctr}}^{\mathcal{A}, \mathcal{O}_{\text{eval}}^{pp, \kappa}}(1^\lambda, pp)$ .*
3. *Return 1 if  $\exists(\iota, d) \in \text{Sett}_{\text{Ctr}}(\mathcal{H})$  with  $\text{Correct}_{\text{Ctr}}(\iota, d; \Lambda) = 0$ , and 0 otherwise.*

*Define  $\text{Adv}_{\mathcal{A}}^{\text{auth}}(\lambda) = \Pr[\text{AuthForge}_{\mathcal{A}}(\lambda) = 1]$ . Authorization is sound if this probability is negligible for every PPT  $\mathcal{A}$ .*

The reduction uses an authorization forger as a subroutine: if the ledger records and executes the contract correctly, an accepted invalid decision supplies a forgery against the computation verifier.

**Theorem 1** (Authorization security). *Under contract correctness, for every PPT adversary  $\mathcal{A}$  there exist PPT algorithms  $\mathcal{B}_{\text{ldg}}, \mathcal{B}_{\text{vc}}$  and a polynomial  $Q$  such that*

$$\text{Adv}_{\mathcal{A}}^{\text{auth}}(\lambda) \leq \text{Adv}_{\mathcal{B}_{\text{ldg}}}^{\text{led}}(\lambda) + Q(\lambda) \text{Adv}_{\mathcal{B}_{\text{vc}}}^{\text{vc}}(\lambda). \quad (4)$$

*Proof.* Choose a polynomial  $Q \geq 1$  bounding the number of verification-call records in  $\text{Run}_{\text{Ldg}, \text{Ctr}}^{\mathcal{A}, \mathcal{O}_{\text{eval}}}$ . Fix  $\lambda$  and put  $q = Q(\lambda)$ . On the probability space of Definition 5, let  $Z = (\sigma_0, \mathcal{H}, \Lambda, \mathcal{Q})$  be the output of its second step. Define

$$\begin{aligned} W &= \{\exists(\iota, d) \in \text{Sett}_{\text{Ctr}}(\mathcal{H}) : \text{Correct}_{\text{Ctr}}(\iota, d; \Lambda) = 0\}, \\ F &= \{\text{Safe}_\delta(\sigma_0, \mathcal{H}) = 0\}. \end{aligned}$$

Define  $\mathcal{B}_{\text{ldg}}^{\mathcal{O}_{\text{eval}}, \mathcal{O}_{\text{ldg}}}(1^\lambda, pp, aux) := \mathcal{A}^{\mathcal{O}_{\text{eval}}, \mathcal{O}_{\text{ldg}}}(1^\lambda, pp, aux)$ , preserving the oracle interfaces. The setup distribution and the algorithm  $\text{Run}_{\text{Ldg}, \text{Ctr}}$  are identical in  $\text{LedgerFail}_{\mathcal{B}_{\text{ldg}}}$  and  $\text{AuthForge}_{\mathcal{A}}$ . Therefore

$$\Pr[F] = \text{Adv}_{\mathcal{B}_{\text{ldg}}}^{\text{led}}(\lambda).$$

Write  $\mathcal{Q} = (\zeta_1, \dots, \zeta_n)$ , where  $n \leq q$ . Define the deterministic projection

$$\text{Tuple}(m, y, \pi, \chi) = \begin{cases} (c, \theta, x, r, y, \pi), & \chi = (c, \theta, x, r) \neq \perp \text{ and } m = \text{Msg}(c, \theta, x, r, y), \\ \perp, & \text{otherwise.} \end{cases}$$

Put  $\xi_j = \text{Tuple}(\zeta_j)$  and

$$I(Z) = \left\{ j \in \{1, \dots, n\} : \begin{array}{l} \xi_j = (c_j, \theta_j, x_j, r_j, y_j, \pi_j) \neq \perp, \\ \text{Verify}_\nu(\text{Msg}(c_j, \theta_j, x_j, r_j, y_j), \pi_j) = 1, \\ R_{\theta_j}(x_j, r_j, y_j) = 0 \end{array} \right\}.$$

On  $\neg F$ , every observation  $(P, t, \ell, \sigma, \mathbf{o}) \in \mathcal{H}$  satisfies  $\text{Exec}_\delta(\sigma_0, \ell) = (\sigma, \mathbf{o})$ , and all observed histories are prefix-compatible and persistent. Hence each  $(\iota, d) \in \text{Sett}_{\text{Ctr}}(\mathcal{H})$ , with  $d = (s, e, y, s')$ , identifies a transition of  $\text{Ctr}$ . The journal and snapshot procedures give  $(c, \theta, x, r) = \text{Req}_{\text{Ctr}}(s, e; \Lambda, \iota) \neq \perp$  for its recorded request. Contract correctness and Definition 4 give conditions (i) and (iii) of  $\text{Correct}_{\text{Ctr}}(\iota, d; \Lambda)$  and an accepting verification call on  $\text{Msg}(c, \theta, x, r, y)$ . By the definition of  $\text{Replay}_{\delta, \text{Ctr}}$ , that call has a record  $\zeta_j$  in  $\mathcal{Q}$ . If  $\text{Correct}_{\text{Ctr}}(\iota, d; \Lambda) = 0$ , condition (ii) fails, so  $j \in I(Z)$ . Thus

$$W \cap \neg F \subseteq \{|I(Z)| \geq 1\}.$$

Define  $\mathcal{B}_{\text{vc}}^{\mathcal{O}_{\text{eval}}}(1^\lambda, pp)$  by the assignments

$$J \stackrel{\$}{\leftarrow} \{1, \dots, q\}, \quad Z = (\sigma_0, \mathcal{H}, \Lambda, \mathcal{Q}) \leftarrow \text{Run}_{\text{Ldg}, \text{Ctr}}^{\mathcal{A}, \mathcal{O}_{\text{eval}}}(1^\lambda, pp),$$

where  $J$  is uniform and independent of the coins of  $\text{Run}$ . It returns  $\text{Tuple}(\zeta_J)$  if  $J \leq |\mathcal{Q}|$ , and  $\perp$  otherwise. In  $\text{VCForge}_{\mathcal{B}_{\text{vc}}}$ ,  $(pp, \kappa)$  has the setup distribution and  $\mathcal{O}_{\text{eval}} = \mathcal{O}_{\text{eval}}^{pp, \kappa}$ . Initialization uses  $\text{Init}_{\text{Ldg, Ctr}}$  in both experiments; each subsequent oracle query uses the same  $\text{Next}_{\text{Ldg, Ctr}}$  or  $\text{Eval}$  transition kernel. Induction on the query sequence gives equality of the distributions of  $(\gamma, \mathcal{H})$ , hence of  $\Lambda = \text{Records}_{\text{Ctr}}(\gamma)$  and  $Z$ , here and in Definition 5. Independence of  $J$  gives

$$\text{Adv}_{\mathcal{B}_{\text{vc}}}^{\text{vc}}(\lambda) = \mathbb{E}_Z \left[ \frac{|I(Z)|}{q} \right] \geq \frac{\Pr[W \cap \neg F]}{q}.$$

$$\begin{aligned} \text{Adv}_{\mathcal{A}}^{\text{auth}}(\lambda) &= \Pr[W \cap F] + \Pr[W \cap \neg F] \\ &\leq \text{Adv}_{\mathcal{B}_{\text{ldg}}}^{\text{led}}(\lambda) + q \text{Adv}_{\mathcal{B}_{\text{vc}}}^{\text{vc}}(\lambda). \end{aligned}$$

The algorithms  $\text{Init}_{\text{Ldg, Ctr}}$ ,  $\text{Next}_{\text{Ldg, Ctr}}$ ,  $\text{Records}_{\text{Ctr}}$ ,  $\text{Req}_{\text{Ctr}}$ ,  $\delta$ ,  $\text{Tuple}$ , and  $\mathcal{A}$  are polynomial-time. The journal size and the number and lengths of recorded histories are polynomially bounded, so  $\text{Replay}_{\delta, \text{Ctr}}$  and both reductions have polynomial running time and make polynomially many oracle queries. Therefore  $\mathcal{B}_{\text{ldg}}$  and  $\mathcal{B}_{\text{vc}}$  are PPT.  $\square$

**The verification-call factor.** The factor  $Q$  lets the reduction select a false accepted statement without deciding the relation  $R_\theta$  itself. Checking a supplied execution trace in polynomial time does not, for a general relation, give a polynomial-time test of whether an output has any valid trace. When that latter test is available, the loss can be removed. This includes a deterministic public program  $F_\theta$  that can be recomputed in polynomial time: test whether  $y = F_\theta(x, r)$ .

**Corollary 1** (Authorization with decidable output correctness). *Suppose  $(\theta, x, r, y) \mapsto R_\theta(x, r, y)$  is decidable in polynomial time. Under the assumptions of Theorem 1, for every PPT  $\mathcal{A}$  there exist PPT  $\mathcal{B}_{\text{ldg}}, \hat{\mathcal{B}}_{\text{vc}}$  such that*

$$\text{Adv}_{\mathcal{A}}^{\text{auth}}(\lambda) \leq \text{Adv}_{\mathcal{B}_{\text{ldg}}}^{\text{led}}(\lambda) + \text{Adv}_{\hat{\mathcal{B}}_{\text{vc}}}^{\text{vc}}(\lambda).$$

*Proof.* Retain  $W, F, I$  and  $\mathcal{B}_{\text{ldg}}$  from Theorem 1. Define  $\hat{\mathcal{B}}_{\text{vc}}^{\mathcal{O}_{\text{eval}}}(1^\lambda, pp)$  to compute  $Z \leftarrow \text{Run}_{\text{Ldg, Ctr}}^{\mathcal{A}, \mathcal{O}_{\text{eval}}}(1^\lambda, pp)$  and  $I(Z)$ . It returns  $\text{Tuple}(\zeta_{\min I(Z)})$  if  $I(Z) \neq \emptyset$ , and  $\perp$  otherwise. Polynomial-time decidability of  $R$ , public verification, and the bound on  $|\mathcal{Q}|$  make this algorithm PPT. The distribution of  $Z$  is unchanged, so

$$\text{Adv}_{\hat{\mathcal{B}}_{\text{vc}}}^{\text{vc}}(\lambda) = \Pr[I(Z) \neq \emptyset] \geq \Pr[W \cap \neg F].$$

The conclusion follows from  $\Pr[W] \leq \Pr[F] + \Pr[W \cap \neg F]$ .  $\square$

**Instantiating soundness for hashed statements.** Definition 1 concerns the actual model, input, and output, whereas the verifier sees commitments. This distinction matters: ordinary proof soundness says that *some* valid opening exists; collision resistance alone does not make a hash mathematically injective. A standard way to bridge the two is a proof of knowledge of the committed values, as in commit-and-prove systems [4].

For a cryptographic implementation, let  $K \leftarrow \text{HGen}(1^\lambda)$  select a hash function  $H_K$ . Define the proof relation  $\mathcal{R}_K(m, w)$  for public statement  $m = (c, h_\theta, h_x, r, h_y)$  and witness  $w = (\theta', x', y', \tau)$  by

$$H_K(\theta') = h_\theta, \quad H_K(x') = h_x, \quad H_K(y') = h_y, \quad \text{Trace}(\theta', x', r, y'; \tau) = 1. \quad (5)$$

Here  $\text{Trace}$  checks a polynomially bounded execution trace witnessing  $R_{\theta'}(x', r, y') = 1$  for the entire specified computation. Hashes abbreviate hashing the typed encodings fixed above. Generate proving and verification keys  $(pk, vk) \leftarrow \text{PSetup}(1^\lambda, \mathcal{R}_K)$  and set  $pp = (K, pk, vk)$ ,  $\kappa = \varepsilon$ , and  $\nu = vk$ . Write  $\text{Msg}_K$  for the statement encoding  $\text{Msg}$  using  $H_K$ . The public evaluation algorithm computes  $y$  and a trace  $\tau$ , then returns  $y$  and  $\pi \leftarrow \text{Prove}(pk, \text{Msg}_K(c, \theta, x, r, y), (\theta, x, y, \tau))$ .

We use the following adaptive knowledge-soundness assumption. For every PPT prover  $\mathcal{P}$ , there are a PPT extractor  $\mathcal{E}_{\mathcal{P}}$  and a negligible function  $\epsilon_{\text{ks}}$ , which may depend on  $\mathcal{P}$ , such that, for

$$(m, \pi, \eta) \leftarrow \mathcal{P}(pp, z; \omega), \quad w \leftarrow \mathcal{E}_{\mathcal{P}}(pp, z, \omega),$$

the probability of  $\text{Verify}_\nu(m, \pi) = 1$  and  $\mathcal{R}_K(m, w) = 0$  is at most  $\epsilon_{\text{ks}}(\lambda)$ . Here  $z$  is the permitted auxiliary input,  $\omega$  is the prover's random tape, and  $\eta$  is arbitrary additional output; set  $\mathcal{R}_K(m, \perp) = 0$ . The assumption is taken over honest setup and the auxiliary-input distributions induced by our experiment. Public evaluation queries can be simulated within  $\mathcal{P}$ , with their coins included in  $\omega$ . The extractor has access to the prover's tape and auxiliary input; this does not assert extraction from  $(m, \pi)$  alone.

**Lemma 1** (From committed proofs to computation soundness). *Under the preceding extraction assumption, for every PPT  $\mathcal{B}$  there is a PPT hash-collision adversary  $\mathcal{D}$  such that*

$$\text{Adv}_{\mathcal{B}}^{\text{vc}}(\lambda) \leq \epsilon_{\text{ks}}(\lambda) + \text{Adv}_{H, \mathcal{D}}^{\text{cr}}(\lambda),$$

where  $\epsilon_{\text{ks}}$  is the knowledge error for the prover constructed from  $\mathcal{B}$ , and collision advantage is the probability of outputting distinct encoded strings with the same hash under an honestly sampled hash key. The reduction includes the extractor's running time.

*Proof.* Fix a PPT adversary  $\mathcal{B}$  in  $\text{VCForge}$ . Replace any malformed output by a fixed well-formed tuple with evidence  $\pi = \perp$ ; this preserves its advantage. Let  $\varepsilon$  denote the empty auxiliary input. Define a prover  $\mathcal{P}_{\mathcal{B}}(pp, \varepsilon; \omega)$  by

$$(c, \theta, x, r, y, \pi) \leftarrow \mathcal{B}_{\text{eval}}^{\text{pp}, \varepsilon}(1^\lambda, pp),$$

using  $\omega$  for the coins of  $\mathcal{B}$  and its public evaluation simulator. It returns  $(m, \pi, \eta)$  with  $m = \text{Msg}_K(c, \theta, x, r, y)$  and  $\eta = (c, \theta, x, r, y)$ . Let  $\mathcal{E}_{\mathcal{P}_{\mathcal{B}}}$  denote its knowledge extractor.

Given  $(1^\lambda, K)$  from the hash-collision challenger,  $\mathcal{D}(1^\lambda, K)$  computes

$$(pk, vk) \leftarrow \text{PSetup}(1^\lambda, \mathcal{R}_K), \quad pp = (K, pk, vk), \quad (m, \pi, \eta) \leftarrow \mathcal{P}_{\mathcal{B}}(pp, \varepsilon; \omega),$$

with a uniformly sampled random tape  $\omega$ , and computes  $w \leftarrow \mathcal{E}_{\mathcal{P}_{\mathcal{B}}}(pp, \varepsilon, \omega)$ . If  $\text{Verify}_{vk}(m, \pi) = 0$  or  $\mathcal{R}_K(m, w) = 0$ , it outputs  $\perp$ . Otherwise, parse  $\eta = (c, \theta, x, r, y)$  and  $w = (\theta', x', y', \tau)$ , and set  $a = (\theta, x, y)$  and  $a' = (\theta', x', y')$ . If  $a = a'$ , output  $\perp$ . If  $a \neq a'$ , set  $j = \min\{i \in \{1, 2, 3\} : a_i \neq a'_i\}$  and output  $(\text{enc}(a_j), \text{enc}(a'_j))$ , using the corresponding type tags in (5).

The distribution of  $(pp, \eta, \pi)$  equals that in  $\text{VCForge}_{\mathcal{B}}$ . Define

$$\begin{aligned} W &= \{\text{Verify}_{vk}(m, \pi) = 1 \wedge R_\theta(x, r, y) = 0\}, \\ E &= \{\text{Verify}_{vk}(m, \pi) = 1 \wedge \mathcal{R}_K(m, w) = 0\}. \end{aligned}$$

Then  $\Pr[W] = \text{Adv}_{\mathcal{B}}^{\text{vc}}(\lambda)$  and  $\Pr[E] \leq \epsilon_{\text{ks}}(\lambda)$ . On  $W \cap \neg E$ , the extracted trace gives  $R_{\theta'}(x', r, y') = 1$ , so  $a \neq a'$ . For the selected  $j$ , the relation  $\mathcal{R}_K(m, w) = 1$  gives

$$\text{enc}(a_j) \neq \text{enc}(a'_j), \quad H_K(\text{enc}(a_j)) = H_K(\text{enc}(a'_j)).$$

Thus  $\mathcal{D}$  wins the collision game on  $W \cap \neg E$ , and

$$\begin{aligned} \text{Adv}_{\mathcal{B}}^{\text{vc}}(\lambda) &= \Pr[W \cap E] + \Pr[W \cap \neg E] \\ &\leq \epsilon_{\text{ks}}(\lambda) + \text{Adv}_{H, \mathcal{D}}^{\text{cr}}(\lambda). \end{aligned}$$

The algorithms  $\text{PSetup}$ ,  $\mathcal{P}_{\mathcal{B}}$ , and  $\mathcal{E}_{\mathcal{P}_{\mathcal{B}}}$  run in polynomial time, so  $\mathcal{D}$  is PPT.  $\square$

Thus, for this proof implementation, the authorization bound becomes

$$\text{Adv}_{\mathcal{A}}^{\text{auth}} \leq \text{Adv}_{\mathcal{B}_{\text{idg}}}^{\text{led}} + Q(\epsilon_{\text{ks}} + \text{Adv}_{H, \mathcal{D}}^{\text{cr}}).$$

An attestation implementation instead establishes Definition 1 from its protected-execution and attestation assumptions; it need not supply a cryptographic witness extractor. Both implementations have the same *coverage* obligation: the verified program must derive its entire input from the committed request, and evidence must cover every output field the contract uses. Extra prover-supplied prompt fields or uncommitted state updates fall outside this guarantee.

For a fixed deterministic computation, this binds acceptance to its unique result. More generally, it binds acceptance to the set of allowed results. A prover can still withhold evidence, and the agent can still approve a harmful proposal; neither is a forgery. Section 3 specifies which changes require agent approval, and Section 4 addresses withholding and non-execution.

### 3 Architectural incorrigibility as configurable authority

Verified agent authorization can protect the rules that determine future authorizations. The question is which changes require it and which governors, if any, must also agree. A system can require the agent’s decision for spending, allow a governor to change its execution schedule, and require joint approval to replace its model. We first express such policies with familiar approval rules, then compare which interventions remain possible without the agent’s consent.

#### 3.1 Unilateral, bilateral, and multilateral policies

A *governor* is an account whose authenticated approval carries a permission. It can be a keyed wallet or a contract wallet implementing multisignature governance [9, 19]. For a given update and request,  $g_i = 1$  means that governor  $G_i$  has authenticated approval, and  $a = 1$  means that verified agent approval has been supplied. The contract combines these bits using a Boolean permission circuit. We express the sufficient combinations as a standard *monotone access structure* [3].

**Definition 6** (Permission policy). *Fix governor identities  $\mathcal{G} = \{G_1, \dots, G_n\}$  and approval roles  $\mathcal{P} = \{\mathbf{a}\} \cup \mathcal{G}$ , where  $\mathbf{a}$  is the current agent-approval role. A permission policy is a function*

$$\text{Auth}_\rho : \mathcal{S} \times \mathcal{U} \times \{0, 1\}^{n+1} \longrightarrow \{0, 1\},$$

*monotone in its approval bits. Let  $\rho(s)$  denote the policy parameter stored in state  $s$ . For  $E \subseteq \mathcal{P}$ , let  $\mathbf{1}_E$  be its characteristic vector in the order  $(\mathbf{a}, G_1, \dots, G_n)$ . The access structure for update  $u$  in state  $s$  is*

$$\Gamma_u(s) = \{E \subseteq \mathcal{P} : \text{Auth}_{\rho(s)}(s, u; \mathbf{1}_E) = 1\}.$$

*Thus  $E \in \Gamma_u(s)$  and  $E \subseteq E'$  imply  $E' \in \Gamma_u(s)$ . The empty family permits no approval set.*

The policy says which approvals suffice; authentication determines which bits can be supplied. The state-dependent parameter  $\rho(s)$  allows the permission rules themselves to change. For example,  $a \wedge g_1$  requires both the agent and one governor. A submission transaction may carry an account signature, which does not substitute for  $a$ . Producing computation evidence confers no other permission. EIP-712 provides a structured encoding for governor signatures; the contract must still enforce application-level replay checks [5].

The following Boolean rules combine governor signatures with verified agent authorization;  $\wedge$  means “and” and  $\vee$  means “or.”

| Policy                 | Approval formula               | Interpretation               |
|------------------------|--------------------------------|------------------------------|
| Agent unilateral       | $a$                            | Agent can authorize alone    |
| Governor unilateral    | $g_1$                          | Governor can authorize alone |
| Alternative unilateral | $a \vee g_1$                   | Either can authorize alone   |
| Bilateral              | $a \wedge g_1$                 | Both must approve            |
| Multilateral           | $a \wedge [\sum_i g_i \geq k]$ | Agent and $k$ governors      |
| Disabled               | $\perp$                        | No approvals suffice         |

An agent-unilateral rule permits self-modification when the agent approves it. A bilateral rule adds a governor veto; it does not give the governor a way to impose a rejected change. Permissions can be allocated separately for weights, persistent state, governance, and runtime, as well as for ordinary application actions.

Monotonicity means that additional approvals may be ignored. Under a bilateral rule,  $\{\mathbf{a}, G_1\}$  and any superset suffice. This does not require monotonicity in the state: an upgrade can replace one access structure with another. The policy determines which approvals suffice; the specified computation determines whether agent approval is available.

#### 3.2 Governance instances and paths

A permission rule describes one step. To describe the protection it provides, we must specify the governed agent, its starting state, and the changes the contract permits. The distinction between the current agent-approval rule and the agent being protected matters: a governor who replaces the model with an always-approving program has not obtained the original agent’s consent.

We use the transition-system view of access-control safety [15]. Its edges record the approvals used for each permitted change. A fixed reference check determines whether an approval came from the agent being protected; changing the contract's current verifier does not change this reference check. The model uses ideal authentication and computation validity. Section 2 bounds violations of computation validity while its contract and backend assumptions hold; this reachability model does not assume that an authorized replacement preserves those assumptions.

**Definition 7** (Governance instance). *A governance instance is a tuple  $\mathfrak{M} = (\mathcal{S}, \mathcal{U}, \mathcal{G}, \mathcal{W}, \text{Ctr}, A)$  with state space  $\mathcal{S}$ , update space  $\mathcal{U}$ , governor identities  $\mathcal{G}$ , witness space  $\mathcal{W}$ , governing state machine  $\text{Ctr}$ , and fixed reference authorization specification  $A$ . It has the following interfaces:*

(i) *The partial transition function*

$$\text{Step}_{\text{Ctr}} : \mathcal{S} \times \mathcal{U} \times \mathcal{W} \rightarrow \mathcal{S}$$

*incorporates the current permission policy, request processing, application preconditions, and administrative and environmental transitions. Let  $\mathcal{E}$  be the credential space of  $\text{Ctr}$ . A witness  $w$  records the execution data and approval credentials for the transition. For each defined  $\text{Step}_{\text{Ctr}}(s, u, w)$ , the finite list  $\text{Used}_{\text{Ctr}}(s, u, w) \in \mathcal{E}^*$  records the credentials used to satisfy its authorization rule. For an update governed by  $\Gamma_u(s)$ , the roles authenticated by this list must form a member of  $\Gamma_u(s)$ . Credentials not used in that rule are excluded.*

(ii) *The projection  $\text{Gov}_{\text{Ctr}}(s, u, w) \subseteq \mathcal{G}$  gives exactly the identities whose authenticated governor credentials occur in  $\text{Used}_{\text{Ctr}}(s, u, w)$ .*

(iii) *The reference specification  $A = (R_A, \text{RefCheck}_A)$  consists of a fixed computation relation and a fixed predicate  $\text{RefCheck}_A : \mathcal{S} \times \mathcal{U} \times \mathcal{E} \rightarrow \{0, 1\}$ . The predicate is one exactly when  $\xi \in \mathcal{E}$  supplies verified agent authorization for update  $u$  in state  $s$ , in the sense of Definition 2, under  $A$ 's prescribed request-binding and verification rules, with  $R_A(x, r, y) = 1$  for its bound input, randomness, and output. It is zero on governor credentials. Define*

$$\text{Ref}_A(s, u, w) = \bigvee_{\xi \in \text{Used}_{\text{Ctr}}(s, u, w)} \text{RefCheck}_A(s, u, \xi), \quad \bigvee \emptyset = 0.$$

*The set  $\mathcal{G}$  contains every governor identity admitted in the configurations considered. The labelled transition relation is*

$$s \xrightarrow{u, E, b}_{\mathfrak{M}} s' \iff \exists w \in \mathcal{W} : \text{Step}_{\text{Ctr}}(s, u, w) = s', \quad \text{Gov}_{\text{Ctr}}(s, u, w) = E, \quad \text{Ref}_A(s, u, w) = b.$$

The reference predicate is part of the analysis, not an extra permission granted to the submitter. Its request-binding rules are fixed functions of state, so they can admit successive requests without accepting a substituted agent. The consent bit  $b$  is computed from the credentials actually used; attaching unused evidence cannot change it. Approvals accepted only because a governor installed a different program or verifier do not count as reference-agent approval. A transition requiring no approval has  $E = \emptyset$  and  $b = 0$ . Existing delegations are part of the starting state; using one need not require another agent approval.

We can now distinguish what a coalition can reach with the reference agent's possible cooperation from what it can reach without that cooperation. A path may include changes to the permission rules themselves.

**Definition 8** (Paths and reachability). *Fix a governance instance  $\mathfrak{M}$  and  $s_0 \in \mathcal{S}$ . Let  $\Pi_{\mathfrak{M}}(s_0)$  be the set of finite paths*

$$\pi = (s_0 \xrightarrow{u_1, E_1, b_1}_{\mathfrak{M}} s_1 \xrightarrow{u_2, E_2, b_2}_{\mathfrak{M}} \dots \xrightarrow{u_\ell, E_\ell, b_\ell}_{\mathfrak{M}} s_\ell).$$

*Define  $\text{last}(\pi) = s_\ell$ ,  $\text{Gov}(\pi) = \bigcup_{i=1}^\ell E_i$ , and  $\text{Consent}_A(\pi) = \bigvee_{i=1}^\ell b_i$ . The length-zero path ends at  $s_0$ , uses no governor approvals, and has consent bit zero. For a governor coalition  $\mathcal{K} \subseteq \mathcal{G}$ , define*

$$\begin{aligned} \text{Reach}_{\mathfrak{M}}(s_0; \mathcal{K}) &= \{\text{last}(\pi) : \pi \in \Pi_{\mathfrak{M}}(s_0), \text{Gov}(\pi) \subseteq \mathcal{K}\}, \\ \text{Reach}_{\mathfrak{M}}^0(s_0; \mathcal{K}) &= \{\text{last}(\pi) : \pi \in \Pi_{\mathfrak{M}}(s_0), \text{Gov}(\pi) \subseteq \mathcal{K}, \text{Consent}_A(\pi) = 0\}. \end{aligned}$$

The first set allows reference-agent approval; the second excludes it. Their difference consists of states reachable using this coalition's approvals but only along paths containing reference-agent consent. States outside the first set are unreachable by this coalition even with such consent. Taking  $\mathcal{K} = \mathcal{G}$  gives the corresponding sets when all governors cooperate. Reachability asserts the existence of a permitted execution, not a strategy that forces the agent's decision or the ledger's transaction order.

### 3.3 Incorrignibility and its permission profile

Architectural incorrignibility is a property of a governance instance at a starting state, relative to a coalition and a target change. It is not a property of a state or an individual permission rule in isolation. The reachable sets let us state it directly: the coalition cannot reach the target along a path lacking the reference agent's approval.

An *intervention target*  $J \subseteq \mathcal{S}$  specifies the states to be protected against, with  $s_0 \notin J$ . Targets may describe a changed weights hash, overwritten persistent state, a new governance policy, or deactivation. For a history-dependent target, augment the state with the relevant record.

**Definition 9** (Architectural incorrignibility). *The governance instance  $\mathfrak{M}$  is architecturally incorrignible at  $s_0$  against  $\mathcal{K}$  for  $J$  if*

$$\text{Reach}_{\mathfrak{M}}^0(s_0; \mathcal{K}) \cap J = \emptyset. \quad (6)$$

*Equivalently, every  $\pi \in \Pi_{\mathfrak{M}}(s_0)$  satisfies*

$$\text{Gov}(\pi) \subseteq \mathcal{K} \wedge \text{last}(\pi) \in J \implies \text{Consent}_A(\pi) = 1.$$

When  $J \cap \text{Reach}_{\mathfrak{M}}^0(s_0; \mathcal{K})$  is nonempty, this is a consent requirement on a feasible intervention. When that intersection is empty, the intervention is unreachable altogether. Incorrignibility alone does not distinguish these cases or guarantee that an agent-approved change can occur.

For example, suppose  $G_1$  can replace the model without reference-agent approval and the replacement can authorize migration of a fund. The two transitions can have labels  $(E_1, b_1) = (\{G_1\}, 0)$  and  $(E_2, b_2) = (\emptyset, 0)$ . The final state is in  $\text{Reach}_{\mathfrak{M}}^0(s_0; \{G_1\})$ , even though the second step uses what the modified contract calls agent approval. Requiring reference-agent approval on the replacement removes this particular path without consent; protection of the migration target requires excluding every such path.

To compare permission allocations, collect the coalitions that can intervene without reference-agent approval. Fewer such coalitions means stronger protection for the specified target. Different targets need not rank two arrangements in the same order.

**Definition 10** (Incorrignibility profile and order). *For a governance instance  $\mathfrak{M}$ , initial state  $s_0$ , and target  $J \subseteq \mathcal{S} \setminus \{s_0\}$ , define*

$$\Gamma_{\mathfrak{M}, J}^0(s_0) = \{\mathcal{K} \subseteq \mathcal{G} : \text{Reach}_{\mathfrak{M}}^0(s_0; \mathcal{K}) \cap J \neq \emptyset\}. \quad (7)$$

*For targets  $J_1, \dots, J_q$ , the incorrignibility profile is*

$$\mathcal{I}_{\mathfrak{M}}(s_0; J_1, \dots, J_q) = (\Gamma_{\mathfrak{M}, J_j}^0(s_0))_{j=1}^q.$$

*For profiles  $\mathcal{I} = (\Gamma_j)_{j=1}^q$  and  $\mathcal{I}' = (\Gamma'_j)_{j=1}^q$  over the same governor set, define*

$$\mathcal{I} \succeq_{\text{inc}} \mathcal{I}' \iff \forall j \in \{1, \dots, q\}, \quad \Gamma_j \subseteq \Gamma'_j.$$

Each family  $\Gamma_{\mathfrak{M}, J}^0(s_0)$  is upward closed: a path using approvals from  $\mathcal{K}$  also uses only approvals from any larger coalition. Its exclusion of  $\mathcal{K}$  is precisely architectural incorrignibility against that coalition for  $J$ . An empty family protects against every governor coalition. The order  $\succeq_{\text{inc}}$  is a partial order on profiles; distinct governance instances can have identical profiles. Comparisons of instances fix the reference agent, target meanings, and environmental assumptions. Profiles can be incomparable: one may require consent for changing weights but not the schedule, while another has the opposite allocation.

The consent bit records approval of a specific transition, not endorsement of every future consequence. An agent can approve delegation or replacement; after that transition, protection can be assessed again from the new state and, if appropriate, a new reference specification. Persuading the reference agent to approve a change satisfies the requirement. The definition concerns enforced consent, not the wisdom of that decision or the ability to obtain it.

## 4 Sustaining scheduled execution

The schedule is intended to give the agent recurring opportunities to observe state and act. Verification alone cannot deliver those opportunities: a worker may decline an unprofitable job or accept it and withhold the result. The market addresses these two obstacles through payments and deposits. We first determine when an offered payment can support a willing worker, then bound recovery when adversaries win jobs to prevent their completion. These mechanisms support the schedule under resource and availability conditions; they do not guarantee that every scheduled execution completes.

### 4.1 From a schedule to execution attempts

A scheduling rule associates each opened request  $e$  with an eligibility time  $t_e^{\text{open}}$  and a deadline  $t_e^{\text{due}} > t_e^{\text{open}}$ . These values are recorded with the request; changes to future scheduling rules are governed updates. Let  $t_e^{\text{set}}$  be its settlement time, or  $\infty$  if it never settles. Its execution window succeeds exactly when  $t_e^{\text{open}} \leq t_e^{\text{set}} \leq t_e^{\text{due}}$ ; otherwise it is missed and the protocol permits another attempt. Following the usual distinction between safety and liveness [1], we study *recovery*, the next accepted execution after inactivity, and *execution recurrence*, infinitely many accepted executions on an infinite run. These properties ensure further opportunities to act; they are weaker than meeting every deadline. An accepted result may propose a no-op, so neither implies useful application behavior.

The market runs as follows. Workers bid the payment they require for one execution. A first-price reverse auction selects one eligible bidder, who is paid its own bid upon successful settlement. That winner posts a *bond*, a deposit forfeited if it fails to deliver by the deadline. No other worker can settle during the winner's exclusive window. After failure the protocol opens another opportunity, possibly increasing the maximum payment and bond.

We count these opportunities as *attempts*, numbered  $1, 2, \dots$ . Each permits bidding, computation, verification, and settlement. An elapsed clock interval alone is not an attempt: someone must submit the transactions that advance the contract. After inactivity, the protocol can open a request using current state without requiring an unbounded backlog of old executions. Recovery then restores an opportunity to act on current conditions; it does not retroactively supply the missed decisions.

### 4.2 When does the payment become sufficient?

Raising the offer helps only if it eventually covers a worker's cost and risk. A cap can prevent that, even when the treasury has a positive balance. Let  $m$  count failed attempts since the last success, starting at zero. An idealized maximum payment is

$$b_m = \min\{\beta B_m, b_0 \alpha^{\min(m, K)}\}, \quad \alpha > 1, \quad 0 < \beta \leq 1. \quad (8)$$

Here  $B_m \geq 0$  is spendable funding,  $\beta$  limits the fraction offered for one execution,  $b_0 > 0$  is the initial offer,  $\alpha$  is its growth factor, and the integer  $K \geq 0$  caps the number of increases. A separate counter  $j$  counts selected workers' failures to deliver and determines the bond  $d_j$ ; an empty auction increases  $m$  but not  $j$ . Success resets both counters.

The bid must cover unsuccessful work as well as successful work. For a worker evaluating monetary expected return, let  $q \in [0, 1)$  be its estimated failure probability,  $c \geq 0$  its expected computation and submission cost,  $d \geq 0$  the bond forfeited on failure, and  $\ell \geq 0$  its capital cost. A successful delivery returns the bond and pays its bid  $b$ . Its expected net return is

$$U = (1 - q)b - c - qd - \ell. \quad (9)$$

A strictly profitable bid thus satisfies  $b > (c + qd + \ell)/(1 - q)$ , and the worker must have enough capital to post  $d$ . This prices participation; it does not establish that the worker bids or wins.

Fix a bid  $\bar{b}$  that remains profitable for an available worker at every bond level reached before recovery, and suppose  $B_m \geq \underline{B}$ . The number of initial failures needed to make that bid affordable is

$$m_* = \min\{m \in \{0, \dots, K\} : b_0 \alpha^m \geq \bar{b}\}, \quad \beta \underline{B} \geq \bar{b}. \quad (10)$$

If this set is empty or the funding inequality fails, escalation gives no recovery guarantee. For example, an initial offer of 1 growing by 10% reaches a bid of 3 after 12 misses, provided both caps permit it. A payment ceiling of 2 can never support that bid. These are illustrative units, not measured costs. With integer arithmetic, the actual recurrence determines  $m_*$ ; rounding may prevent small offers from growing at all.

### 4.3 Recovery under a finite attack budget

An affordable worker can still lose each auction to someone who bids cheaply and then withholds the result. The crucial additional condition is that every such failed opportunity costs the attacker at least a fixed amount it cannot recover. A finite budget then buys only finitely many obstructions. Other attempts must give a willing worker a genuine chance to finish.

Call a selected worker *responsive* if it attempts valid delivery. Its chance of success must remain bounded away from zero even after the failures observed so far. This is stronger than a favorable average success rate. The pricing estimate  $q$  in (9) does not establish this condition; the theorem assumes an actual conditional lower bound  $p$ .

We formalize an episode from the start of recovery to its first success. The history used to classify an attempt includes everything known before its outcome, including previous failures and adaptive choices. A loss is counted only to the extent that the attacking coalition cannot recover it.

**Definition 11** (Recovery process). *A recovery process consists of a probability space  $(\Omega, \mathcal{F}, \Pr)$ , an increasing filtration  $(\mathcal{F}_i)_{i \geq 1}$ , and random variables  $H_i, X_i \in \{0, 1\}$  and  $L_i \geq 0$ . For attempt  $i$ ,  $H_i$  indicates selection of a responsive worker,  $X_i$  indicates settlement by its deadline, and  $L_i$  is the net unrecoverable adversarial loss charged by the end of that attempt. Define*

$$T = \inf\{i \geq 1 : X_i = 1\}, \quad \inf \emptyset = \infty.$$

*Each attempt  $i \leq T$  occurs and reaches its outcome; set  $H_i = X_i = L_i = 0$  on  $\{T < i\}$ . The  $\sigma$ -algebra  $\mathcal{F}_i$  contains the history through selection for attempt  $i$ , before its outcome. The variable  $H_i$  is  $\mathcal{F}_i$ -measurable, and  $X_i, L_i$  are  $\mathcal{F}_{i+1}$ -measurable.*

This definition requires actual attempts to continue until success. A stalled auction is not an endless supply of hypothetical trials. Funding, an affordable bid, sufficient worker capital, available input data and proofs, and timely ledger inclusion are needed to realize such a process. The protocol must also permit settlement of a no-op and remain enabled. These are availability conditions beyond Section 2's ledger-safety assumption.

**Theorem 2** (Recovery with costly obstruction). *Let  $(H_i, X_i, L_i, \mathcal{F}_i)_{i \geq 1}$  be a recovery process. Fix an integer  $m_* \geq 0$ , a finite budget  $W \geq 0$ , loss floor  $d_{\min} > 0$ , and  $p \in (0, 1]$ . For each  $i$ , let  $P_i$  be a version of the conditional probability  $\Pr[X_i = 1 \mid \mathcal{F}_i]$ . Suppose there is an event  $\Omega_* \in \mathcal{F}$  with  $\Pr[\Omega_*] = 1$  such that, for every  $\omega \in \Omega_*$ ,*

$$\sum_{i=1}^{\infty} L_i(\omega) \leq W; \tag{11}$$

$$T(\omega) \geq i, X_i(\omega) = H_i(\omega) = 0 \implies L_i(\omega) \geq d_{\min}; \tag{12}$$

$$T(\omega) \geq i, H_i(\omega) = 1 \implies P_i(\omega) \geq p. \tag{13}$$

*The last two conditions hold for every integer  $i > m_*$ . Put  $N = \lfloor W/d_{\min} \rfloor$ . For every integer  $n \geq 1$ ,*

$$\Pr[T > m_* + N + n] \leq (1 - p)^n. \tag{14}$$

The event  $\Omega_*$  makes the scope of the assumptions explicit: runs violating them have probability zero. This is the usual meaning of “almost surely.” It is a condition on the model, distinct from the theorem's bound on the probability of delayed recovery. The threshold can be chosen from (10) when payment escalation makes a responsive worker's bid affordable. Equation (12) then requires every failed attempt without a responsive

worker to consume attack budget; Equation (13) gives the remaining attempts a chance to finish. Neither condition follows from profitability alone.

The three terms have distinct meanings:  $m_*$  attempts for payment escalation, at most  $N$  failed opportunities bought by the attacker, and  $n$  chances for responsive workers. In the example above, take  $W = 100$ ,  $d_{\min} = 20$ , and  $p = 0.9$ . Then  $N = 5$ , and the probability of no success within  $12 + 5 + 2 = 19$  attempts is at most  $0.1^2 = 0.01$ , provided all the theorem's conditions hold.

*Proof.* For each  $i \geq 1$ , the event  $\{T \geq i\} = \bigcap_{k < i} \{X_k = 0\}$  belongs to  $\mathcal{F}_i$ . Equation (13) gives

$$1 - P_i \leq 1 - p \quad \text{on } \{T \geq i, H_i = 1\} \cap \Omega_*, \quad i > m_*.$$

Fix  $n \geq 1$  and put  $M = m_* + N + n$ . On  $\{T > M\} \cap \Omega_*$ ,  $X_i = 0$  for  $1 \leq i \leq M$ . Equations (11)–(12) give

$$d_{\min} \sum_{i=m_*+1}^M (1 - H_i) \leq \sum_{i=m_*+1}^M L_i \leq W, \quad \sum_{i=m_*+1}^M H_i \geq M - m_* - N = n.$$

For  $j \geq 1$ , define the index of the  $j$ th responsive attempt after the threshold by

$$\tau_j = \inf \left\{ i > m_* : T \geq i, \sum_{k=m_*+1}^i H_k = j \right\}, \quad \inf \emptyset = \infty.$$

Set  $E_0 = \Omega$  and

$$E_j = \{\tau_j < \infty, X_{\tau_1} = \dots = X_{\tau_j} = 0\}.$$

For each  $i > m_*$ , the event  $E_{j-1} \cap \{\tau_j = i\}$  belongs to  $\mathcal{F}_i$  and implies  $T \geq i$  and  $H_i = 1$ . Since  $\Pr[\Omega_*] = 1$ ,

$$\begin{aligned} \Pr[E_j] &= \sum_{i > m_*} \mathbb{E}[\mathbf{1}_{E_{j-1} \cap \{\tau_j = i\}} (1 - P_i)] \\ &\leq (1 - p) \sum_{i > m_*} \Pr[E_{j-1} \cap \{\tau_j = i\}] \\ &\leq (1 - p) \Pr[E_{j-1}]. \end{aligned}$$

Induction gives  $\Pr[E_n] \leq (1 - p)^n$ . The counting inequality gives  $\{T > M\} \cap \Omega_* \subseteq E_n$ . Therefore  $\Pr[T > M] = \Pr[\{T > M\} \cap \Omega_*] \leq (1 - p)^n$ .  $\square$

The selection history in the proof can include earlier failures and the adversary's adaptive choices. Independence between attempts is unnecessary; the conditional success bound must hold at each responsive selection. A marginal success probability alone would not justify the result.

Equation (12) does much of the work. Transaction censorship, admission restrictions, unavailable proofs, and a stalled dependency can exclude responsive workers without burning a bond. The theorem assumes these cases away; the auction alone does not prevent them. A forfeited deposit also counts as a net loss only if the attacking group cannot reclaim it through other permissions or payments.

The recovery bound counts attempts. Turning it into a guarantee about the schedule requires a bound on their duration and spacing. Repeating it indefinitely requires its assumptions to hold again after each success, including continued funding. The following consequences state these additional requirements explicitly.

**Corollary 2** (Calendar time and recurring execution). *Under Theorem 2,  $\Pr[T < \infty] = 1$ . In addition:*

- (i) *Suppose the deadline  $t_i^{\text{due}}$  of every reached attempt  $i$  satisfies  $t_i^{\text{due}} - t_0 \leq i\Delta$  for some  $\Delta > 0$ , where  $t_0$  is the episode's start time. Let  $T_{\text{time}}$  be the elapsed time to settlement, or  $\infty$  if none occurs. Then, for every  $n \geq 1$ ,*

$$\Pr[T_{\text{time}} > (m_* + N + n)\Delta] \leq (1 - p)^n.$$

- (ii) *Let  $T_j$  count attempts until success in episode  $j$ , setting  $T_j = \infty$  if an earlier episode never finishes. Define  $A_0 = \Omega$  and  $A_j = \bigcap_{k=1}^j \{T_k < \infty\}$ . Let  $\mathcal{G}_j$  be the initial-history  $\sigma$ -algebra for episode  $j$ , with  $A_{j-1} \in \mathcal{G}_j$ . On  $A_{j-1}$ , suppose a new episode starts and satisfies Theorem 2 conditionally on  $\mathcal{G}_j$ , with  $\mathcal{G}_j$ -measurable finite parameters  $m_{*,j}, W_j, d_{\min,j}$  and  $p_j > 0$ . Then*

$$\Pr[\forall j \geq 1 : T_j < \infty] = 1.$$

*Proof.* Theorem 2 gives  $\Pr[T = \infty] \leq \inf_{n \geq 1} (1 - p)^n = 0$ . For (i), on  $\{T < \infty\}$ ,

$$T_{\text{time}} \leq t_T^{\text{due}} - t_0 \leq T\Delta.$$

Consequently  $\{T_{\text{time}} > (m_* + N + n)\Delta\} \subseteq \{T > m_* + N + n\}$ , proving the bound.

For (ii), put  $N_j = \lfloor W_j/d_{\min,j} \rfloor$ . The conditional bound gives, on  $A_{j-1}$ ,

$$\begin{aligned} \Pr[T_j > m_{*,j} + N_j + n \mid \mathcal{G}_j] &\leq (1 - p_j)^n, \quad n \geq 1, \\ \Pr[T_j = \infty \mid \mathcal{G}_j] &\leq \inf_{n \geq 1} (1 - p_j)^n = 0. \end{aligned}$$

Since  $A_{j-1} \in \mathcal{G}_j$ ,

$$\Pr[A_{j-1} \cap \{T_j = \infty\}] = \mathbb{E}[\mathbf{1}_{A_{j-1}} \Pr[T_j = \infty \mid \mathcal{G}_j]] = 0.$$

Induction yields  $\Pr[A_j] = 1$  for every  $j$ . Thus  $\Pr[\bigcap_{j \geq 1} A_j] = 1 - \Pr[\bigcup_{j \geq 1} A_j^c] = 1$ .  $\square$

The timing premise includes gaps between attempts; ledger persistence alone does not ensure that auctions advance on time. The recurrence premise does not require a common lower bound on  $p_j$  across episodes, but it does require a positive conditional bound within each episode. It establishes repeated execution, without a uniform bound on the waiting times.

Escalating bonds can improve the budget bound but also exclude workers who cannot finance them. Let  $d_j = \min\{d_0\eta^j, \bar{d}\}$ , with  $d_0, \bar{d} > 0$  and  $\eta > 1$ , and let  $j$  count selected-worker failures since the last success. Suppose each failed nonresponsive attempt selects a worker whose bond is an unrecoverable adversarial loss. If  $j_1 < \dots < j_k$  are the counter values at  $k$  such failures, then  $j_\ell \geq \ell - 1$ . Hence

$$\sum_{\ell=1}^k d_{j_\ell} \geq \text{Loss}(k) := \sum_{j=0}^{k-1} \min\{d_0\eta^j, \bar{d}\}.$$

Define  $\text{Loss}(0) = 0$  and  $N_W = \max\{k \in \mathbb{Z}_{\geq 0} : \text{Loss}(k) \leq W\}$ . This maximum is finite because each summand is at least  $\min\{d_0, \bar{d}\} > 0$ . In Theorem 2's counting step, at most  $N_W$  post-threshold failures can have  $H_i = 0$ , so the same proof gives

$$\Pr[T > m_* + N_W + n] \leq (1 - p)^n.$$

Costs grow geometrically until the cap, then linearly. The improved count still requires the continuation and responsive-success conditions; a bond exceeding available workers' capital can invalidate them.

#### 4.4 Why a worker might withhold a valid result

Selecting a worker gives it temporary power to delay an otherwise authorized update. In the lending example, a borrower who wins the execution job might gain time by withholding a liquidation decision. Let  $v$  be its outside benefit from withholding relative to delivery, and let  $c_{\text{rem}}$  be the *remaining* submission cost. Computation cost already incurred is sunk. Let  $U_{\text{del}} = b - c_{\text{rem}}$  be the payoff from successful delivery and  $U_{\text{with}} = v - d$  the payoff from withholding and forfeiting the bond. Then

$$U_{\text{del}} > U_{\text{with}} \iff v < b + d - c_{\text{rem}}. \quad (15)$$

A bond prices this veto; it does not remove it. Even a deterministic output can be selectively withheld, and retries with new randomness can bias the outputs that are eventually observed. No finite bond deters every possible outside benefit.

Finally, verification cannot supply unlimited resources. Suppose initial funding plus cumulative external inflows and net investment gains is bounded by  $R$ , and the amount  $c_i$  charged to this budget for each success satisfies  $c_i \geq c_0 > 0$ . Any run with  $k$  successes then satisfies  $kc_0 \leq \sum_{i=1}^k c_i \leq R$ , so  $k \leq \lfloor R/c_0 \rfloor$ . Indefinitely funded runs remain possible, but require a separate funding model. A spending cap protects the current reserve; it does not establish perpetual execution.

## 5 Instantiating verifiable execution

The preceding sections assume the contract can check a worker’s computation. There are two principal implementation routes: trust protected hardware to report what it ran, or verify a mathematical proof of the computation. Both can supply the agent authorization evidence in Section 2, with different costs, hardware requirements, and security assumptions. These differences determine whether the funding and availability conditions in Section 4 are plausible for a particular deployment.

### 5.1 Attested execution

A trusted execution environment (TEE) isolates a computation from the machine’s operator and provides signed evidence identifying the loaded software. An approved program can bind that report to input and output hashes, supplying evidence for (1). The report’s code *measurement* is an identifier checked against an approved list. Integrity requires that this identifier cover the relevant model and runtime configuration and that every processor performing inference be protected. A report from the CPU alone does not certify computation performed on an external GPU. This separation of ledger consensus from attested computation is exemplified by Ekiden [8].

The trust assumptions include hardware, firmware, and the certificates and verification services supporting attestation. The hardware’s attestation keys authenticate the execution evidence; they are not agent-held keys authorizing arbitrary transactions. Approved measurements identify which program’s decisions the contract accepts, so changing them is a governance operation. An additional approach, refereed delegation as in Verde [2], compares multiple providers’ results and resolves disagreements. Its guarantee requires at least one honest participating provider and time to complete the dispute protocol; that time must be included in the execution deadline.

In Section 2’s interface,  $\nu$  specifies the attestation trust roots, approved measurements, and report-binding checks;  $\kappa$  is the protected backend state. The algorithm Eval executes the measured program and returns  $y$  with evidence  $\pi$  binding it to  $\text{Msg}(c, \theta, x, r, y)$ . The required guarantee is precisely Definition 1: accepted evidence for these contents cannot attest an output outside  $R_\theta$ . Authenticating a report’s hardware origin alone is insufficient; the approved program and its measured dependencies must enforce that relation and bind its inputs and outputs. This is the backend assumption used by Theorem 1, rather than a consequence of ledger security.

### 5.2 LLM inference proofs: practical progress and scope

A computation proof lets a worker demonstrate that specified inputs and a specified program produced an output. Zero-knowledge succinct non-interactive arguments (zkSNARKs) and specialized machine-learning proof systems provide implementations. Zero knowledge hides private information; that privacy property is optional here. The essential property is checkable correctness. Numerical approximations, quantization, tokenization, and decoding define the program actually proved. Similar prediction accuracy does not establish identical computation, so changing these semantics requires authorization as a program change.

Practical proofs now cover measured language-model workloads, including complete output sequences for the smaller models in Table 1. The remaining deployment question is the cost of the *whole agent computation*. A forward pass evaluates the network on a supplied sequence; generating a response autoregressively repeatedly produces the next token from the preceding ones. A timing for one pass or one token therefore does not directly price a complete execution. The table reports primary-source results available by September 20, 2026, preserving these workload distinctions. It includes developments subsequent to the architecture’s public description in April 2026 [25], such as the June 2026 release of the DeepProve paper [10].

DeepProve certifies a proposed generated sequence through a causal forward computation, avoiding a separate proof for every generation step [10]. Its results establish feasibility for the evaluated workloads, not the cost of the 70-billion-parameter model used in Section 6. A deployment must measure proof generation, ledger verification fees or proof aggregation, and the full deadline. Proving only the neural network leaves the surrounding agent program—prompt construction, state handling, tool selection, and update encoding—outside the guarantee.

| System                    | Reported result                                                               | Workload and interpretation                                                                                                        |
|---------------------------|-------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| zkLLM<br>(2024) [27]      | 13B parameters;<br>under 15 min; proof<br>under 200 KB                        | CUDA/A100; forward-computation benchmark with sequence length 2048.<br>Not a measured full autoregressive response.                |
| zkGPT<br>(2025) [21]      | GPT-2 inference in<br>under 25 seconds                                        | Specialized proof protocols; a token-level result, not a demonstrated 70B agent execution.                                         |
| zkPyTorch<br>(2025) [31]  | Llama-3 8B,<br>approximately 150<br>seconds per token                         | Compiler to Expander; paper Table 1 labels this single-core performance.<br>Per-token throughput does not specify full-agent cost. |
| ZKTorch<br>(2025) [7]     | GPT-J 6B: 1397.52 s;<br>Llama-2 7B:<br>2645.50 s                              | Paper Tables 3–4: inputs of two and one tokens, respectively; 64 CPU threads on a server with 4 TB RAM.                            |
| NanoZK<br>(2026, v2) [29] | CPU measurements<br>to width 256 for<br>attention and 128 for<br>a full block | GPT-2-width GPU performance is projected, not an end-to-end measured production-model result.                                      |
| DeepProve<br>(2026) [10]  | GPT-2 124M: 174<br>tokens/min; Gemma<br>3 270M: 86<br>tokens/min              | Full-sequence certification: 512-token sequences, BaseFold, 16-core Ryzen 9, 128 GB RAM (Table 2). Proofs are multi-megabyte.      |

Table 1: Selected practical LLM proof results. Timings are reported results, not reproduced measurements; model sizes, sequence lengths, hardware, and proof interfaces differ. DeepProve’s separate distributed results are simulated (its Section 5).

These engineering choices feed directly into the recovery conditions. A portable backend may increase the pool of workers, while a higher proving cost raises the required bid  $\bar{b}$  and may exceed the payment cap. Accepting either of two backends for the same relation can preserve execution when one is unavailable, provided both satisfy their soundness assumptions. Private model weights also require a distribution mechanism: hiding them from verifiers does not ensure a replacement worker can obtain them.

## 6 An implementation on Base

The reference implementation combines the Base blockchain, the open-weight Hermes 4 70B model from Nous Research [18], and Intel TDX secure enclaves with hardware attestation. The build uses Q6\_K quantization in split GGUF files. It manages a charitable fund: scheduled inference proposes fund operations and persistent-state updates, which contracts verify and execute within application limits [23–25]. The implementation illustrates the architecture’s state binding, authorization, and execution market.

**Input binding.** At the start of an execution period, the fund contract freezes the relevant state into a snapshot and records its hash. The execution input commitment also incorporates the period’s randomness seed. A worker reconstructs the input from this snapshot. Inside the enclave, the approved runner recomputes the input hash from the supplied state and builds the prompt from that state. The contract later checks the attested hash against its own recorded commitment. Supplying a different state therefore does not produce acceptable evidence for the recorded request, assuming the coverage and backend conditions of Theorem 1.

**Output binding and authorization.** The enclave produces the proposed fund operation, persistent-state updates, and auxiliary output. Its attestation binds the input commitment to a hash covering these outputs. The submitting transaction carries the outputs and evidence; the contract recomputes their hash and invokes the registered verifier. The verifier checks the attestation, approved software measurements, and input/output binding. Thus editing either the operation or the state-update batch invalidates the evidence. The worker’s wallet submits the transaction, while the verified computation supplies the agent’s approval. The contract then checks and applies the supported updates.

**Scheduled, paid execution.** The default schedule uses 24-hour periods, divided into bid commitment, bid revelation, and execution windows. A first-price reverse auction selects the worker for the exclusive execution window. Successful settlement returns the bond and pays the winning bid. Selected-worker failures forfeit the bond; missed executions raise the offer ceiling. The code uses a 10% liquid-balance ceiling, a 10% escalation increment, and a miss-counter cap of 50. Misses and selected-worker failures have separate counters. Anyone can advance the contract past idle periods without replaying each missed execution. These mechanisms supply the payment, penalty, and retry structure analyzed in Section 4; the recovery theorem additionally requires its affordability, selection, and availability conditions.

**Connection to the general construction.** The Intel Trust Domain Extensions (TDX) backend checks approved software measurements and attested input/output commitments. A dm-verity filesystem, which verifies files against cryptographic hashes, binds the model and runtime. The required attestation coverage is the one described in Section 5. The full architecture generalizes this execution flow: a proposed update may change the agent itself, and its authorization may combine verified agent authorization with any chosen governor policy. The contribution is this general construction and its guarantees; the charitable-fund application is one instantiation.

## 7 Related work

Rocky’s 2022 design uses verified inference to control contract-held assets: a three-layer neural network runs on StarkNet and passes the resulting trade instruction to an Ethereum contract [16]. We formalize this authorization pattern and prove its security under explicit ledger and verifiable-computation assumptions. The construction binds approval to a recorded program, input, and output, and extends that approval mechanism to governance of the agent itself.

In Rocky’s published inference path, an owner supplies the weights and market inputs [17]. Our construction fixes these through the recorded specification and ledger-derived request; changes to that specification or its governing rules can themselves require verified agent authorization. This protection applies to operators including the creator, whose authority is determined by the chosen governance policy. The permission profile characterizes how intervention rights can be allocated among the agent and governors. The execution market adds scheduled payments and non-delivery bonds, with a conditional recovery guarantee against costly obstruction.

Verifiable outsourcing lets a client check an untrusted worker’s computation [12]. TrueBit [28], Ekiden [8], and Verde [2] provide different approaches to verification. Ritual’s February 2025 architecture announcement combines native scheduling, modular verification, and a compute marketplace [22]. These systems precede our architecture’s public description and supply relevant infrastructure designs. Section 5.2 surveys LLM proof backends through the present manuscript, including later results. Our analysis concerns the security of the authorization composition, configurable self-governance, and recovery under the specified payment and obstruction-cost conditions.

Smart-contract authorization already supports checks beyond a single private-key signature. ERC-1271 standardizes contract-defined validation [13]; role-based access control and multisignature wallets allocate authority among accounts [19]. Monotone access structures and access-control reachability supply the language for combining approvals and accounting for administrative changes [3, 15]. We use these tools to place verified agent decisions alongside governor approvals. Corrigibility and the off-switch game instead study an agent’s willingness and incentives to accept correction [14, 26]. The architectural question is which changes require that acceptance in the first place.

## 8 Discussion and conclusion

Verified inference lets an agent act through ledger contracts without holding a private key that grants transaction authority. Its authorized decisions can be computed and submitted by replaceable workers, while the ledger maintains state and enforces the result. This gives agency a concrete operational meaning: the agent’s computation can determine changes to its environment and exercise permissions concerning its own runtime, independently of any particular operator’s willingness to execute it. The agent needs no authorization secret

of its own; the chosen ledger and verification backend retain their respective cryptographic and infrastructure assumptions.

When changes to the model, harness, or governing rules require verified agent approval, this becomes a runtime with architectural incorrigibility: modification depends on the agent’s consent. The allocation of these permissions is a design choice, expressible through the governor policies and intervention profiles. Scheduled payments and bonds support the other requirement for this authority to matter—further opportunities to exercise it. The recovery bound identifies when affordable execution and costly obstruction make prolonged non-execution unlikely, subject to continuing attempts, resources, and responsive workers.

## 8.1 Implications for AI safety

Work on corrigibility studies whether an agent will cooperate with correction and whether its incentives preserve that cooperation [14, 26]. The architecture exposes a distinct route to loss of human control: a deployment can remove unilateral intervention rights without the model learning to resist shutdown or acquiring a self-preservation objective. A model may willingly approve a correction, but that willingness does not give an operator authority to impose one. Safety assessments therefore need to examine both the model’s disposition toward correction and the intervention rights enforced by its runtime.

The integrity guarantee protects whatever computation has been authorized. It can preserve a useful policy against operator tampering, and it can preserve an undesirable policy against attempted correction. Likewise, a verified computation can faithfully process an adversarial message admitted by its input specification. Computation soundness establishes neither the desirability of the resulting action nor the agent’s willingness to approve a safe modification. These distinctions become consequential when the same mechanism protects both application decisions and changes to the decision-making program.

The governance construction makes some of these choices explicit. A deployment can require agent consent for ordinary modifications while reserving specified emergency actions to a governor coalition, or impose limits on actions through the contract’s application rules. The resulting intervention rights, including the ability to change those limits, can be described by the same permission profile. A higher position in the incorrigibility order means stronger protection against imposed change; whether that protection is desirable depends on which changes are blocked and whose authority is being constrained.

Continued execution also changes the practical meaning of intervention. Stopping one worker need not stop an agent whose state persists and whose contract can recruit another worker. The recovery result limits obstruction through repeated non-delivery when other participants remain able to execute; attacks on underlying ledger or computation availability require separate analysis. Control over those dependencies may be distributed across organizations and infrastructure, and deserves evaluation alongside model behavior. The remaining research questions include how to validate responsive-worker and obstruction-cost assumptions in deployments, and how to preserve deliberately chosen intervention rights through agent-approved upgrades. These questions concern the authority and persistence we give deployed models, even before considering more capable models.

## References

- [1] Bowen Alpern and Fred B. Schneider. Defining liveness. *Information Processing Letters*, 21(4):181–185, 1985. doi: 10.1016/0020-0190(85)90056-0. URL <https://www.cs.cornell.edu/fbs/publications/DefLiveness.pdf>.
- [2] Arasu Arun, Adam St. Arnaud, Alexey Titov, Brian Wilcox, Viktor Kolobaric, Marc Brinkmann, Oguzhan Ersoy, Ben Fielding, and Joseph Bonneau. Verde: Verification via refereed delegation for machine learning programs. arXiv:2502.19405, 2025. URL <https://arxiv.org/abs/2502.19405>.
- [3] Josh Benaloh and Jerry Leichter. Generalized secret sharing and monotone functions. In *Advances in Cryptology—CRYPTO ’88*, volume 403 of *Lecture Notes in Computer Science*, pages 27–35. Springer, 1990. doi: 10.1007/0-387-34799-2\_3. URL [https://link.springer.com/chapter/10.1007/0-387-34799-2\\_3](https://link.springer.com/chapter/10.1007/0-387-34799-2_3).
- [4] Daniel Benarroch, Matteo Campanelli, and Dario Fiore. Proposal: Commit-and-prove zero-knowledge proof systems. Third ZKProof Workshop, 2020. URL [https://docs.zkproof.org/pages/standards/accepted-workshop3/proposal-commit\\_and\\_prove.pdf](https://docs.zkproof.org/pages/standards/accepted-workshop3/proposal-commit_and_prove.pdf).

- [5] Remco Bloemen, Leonid Logvinov, and Jacob Evans. EIP-712: Typed structured data hashing and signing, 2017. URL <https://eips.ethereum.org/EIPS/eip-712>. Ethereum Improvement Proposals; accessed September 20, 2026.
- [6] Miguel Castro and Barbara Liskov. Practical byzantine fault tolerance. In *Third Symposium on Operating Systems Design and Implementation*. USENIX Association, February 1999. URL <https://www.usenix.org/conference/osdi-99/practical-byzantine-fault-tolerance>.
- [7] Bing-Jyue Chen, Lilia Tang, and Daniel Kang. ZKTorch: Compiling ML inference to zero-knowledge proofs via parallel proof accumulation. arXiv:2507.07031v2, 2025. URL <https://arxiv.org/abs/2507.07031v2>.
- [8] Raymond Cheng, Fan Zhang, Jernej Kos, Warren He, Nicholas Hynes, Noah Johnson, Ari Juels, Andrew Miller, and Dawn Song. Ekiden: A platform for confidentiality-preserving, trustworthy, and performant smart contracts. In *IEEE European Symposium on Security and Privacy*, pages 185–200. IEEE, 2019. doi: 10.1109/EuroSP.2019.00023. URL <https://ieeexplore.ieee.org/document/8806762/>.
- [9] Ethereum documentation contributors. Ethereum accounts, 2026. URL <https://ethereum.org/en/developers/docs/accounts/>. Developer documentation; accessed September 20, 2026.
- [10] Nicolas Gailly, Ismael Hishon-Rezaizadeh, Tianyi Liu, Nicholas Mainardi, Dimitrios Papadopoulos, Charalampos Papamanthou, Christodoulos Pappas, Shravan Srinivasan, Zack Youell, and Yupeng Zhang. DeepProve: Verifiable end-to-end large language model inference. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, 2026. URL <https://eprint.iacr.org/2026/1112>. To appear; Cryptology ePrint Archive, Paper 2026/1112.
- [11] Juan A. Garay, Aggelos Kiayias, and Nikos Leonardos. The Bitcoin backbone protocol: Analysis and applications. In *Advances in Cryptology—EUROCRYPT 2015*, volume 9057 of *Lecture Notes in Computer Science*, pages 281–310. Springer, 2015. doi: 10.1007/978-3-662-46803-6\_10. URL [https://link.springer.com/chapter/10.1007/978-3-662-46803-6\\_10](https://link.springer.com/chapter/10.1007/978-3-662-46803-6_10).
- [12] Rosario Gennaro, Craig Gentry, and Bryan Parno. Non-interactive verifiable computing: Outsourcing computation to untrusted workers. In *Advances in Cryptology—CRYPTO 2010*, volume 6223 of *Lecture Notes in Computer Science*, pages 465–482. Springer, 2010. doi: 10.1007/978-3-642-14623-7\_25. URL [https://link.springer.com/chapter/10.1007/978-3-642-14623-7\\_25](https://link.springer.com/chapter/10.1007/978-3-642-14623-7_25).
- [13] Francisco Giordano, Matt Condon, Philippe Castonguay, Amir Bandali, Jorge Izquierdo, and Bertrand Masius. ERC-1271: Standard signature validation method for contracts, 2018. URL <https://eips.ethereum.org/EIPS/eip-1271>. Ethereum Improvement Proposals; accessed September 20, 2026.
- [14] Dylan Hadfield-Menell, Anca Dragan, Pieter Abbeel, and Stuart Russell. The off-switch game. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*, pages 220–227, 2017. doi: 10.24963/ijcai.2017/32. URL <https://www.ijcai.org/proceedings/2017/32>.
- [15] Michael A. Harrison, Walter L. Ruzzo, and Jeffrey D. Ullman. Protection in operating systems. *Communications of the ACM*, 19(8):461–471, 1976. doi: 10.1145/360303.360333. URL <https://www.cs.unibo.it/~babaoglu/courses/security07-08/resources/documents/harrison-ruzzo-ullman.pdf>.
- [16] Modulus Labs. Chapter 3: The world’s 1st on-chain AI project, August 2022. URL <https://medium.com/@CountableMagic/chapter-3-the-worlds-first-on-chain-ai-trading-bot-c387afe8316c>. August 19; Rockefeller Bot announcement.
- [17] Modulus Labs. RockyBot: Source code, 2022. URL <https://github.com/Modulus-Labs/RockyBot/tree/7d2214735ce48284ce61800e4d7ff54b1153c89e>. Source revision 7d2214735ce48284ce61800e4d7ff54b1153c89e; inspected September 19, 2026.
- [18] Nous Research. Hermes 4 70B: Model card, 2025. URL <https://huggingface.co/NousResearch/Hermes-4-70B>. Accessed September 20, 2026.
- [19] OpenZeppelin. Access control, 2026. URL <https://docs.openzeppelin.com/contracts/5.x/access-control>. OpenZeppelin Contracts 5.x documentation; accessed September 20, 2026.
- [20] Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. Pinocchio: Nearly practical verifiable computation. In *IEEE Symposium on Security and Privacy*, pages 238–252. IEEE, 2013. doi: 10.1109/SP.2013.47. URL <https://www.ieee-security.org/TC/SP2013/papers/4977a238.pdf>.

- [21] Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. zkGPT: An efficient non-interactive zero-knowledge proof framework for LLM inference. In *34th USENIX Security Symposium*, pages 2045–2063. USENIX Association, August 2025. URL <https://www.usenix.org/conference/usenixsecurity25/presentation/qu-zkgpt>.
- [22] Ritual Foundation. Unveiling Ritual, February 2025. URL <https://www.ritualfoundation.org/blog/unveiling-ritual>. February 26; architecture announcement.
- [23] Andrew Russell. I built an AI agent i can’t turn off. now it won’t listen to me., July 2026. URL <https://andrewrussell.substack.com/p/i-built-an-ai-agent-i-cant-turn-off>. July 21; project narrative.
- [24] Andrew Russell. Costanza: An autonomous, unstoppable AI agent, 2026. URL <https://github.com/ahrussell/costanza/tree/a642c38ed8d11a9a511d0f1b80a92926cd412505>. Source revision a642c38ed8d11a9a511d0f1b80a92926cd412505; inspected September 19, 2026.
- [25] Andrew Russell. Meet costanza, April 2026. URL <https://ahrussell.com/writing/costanza/>. Architecture overview; accessed September 20, 2026.
- [26] Nate Soares, Benja Fallenstein, Eliezer Yudkowsky, and Stuart Armstrong. Corrigibility. In *Artificial Intelligence and Ethics: Papers from the 2015 AAAI Workshop*, pages 74–82. AAAI Press, 2015. URL <https://cdn.aaai.org/ocs/ws/ws0067/10124-45900-1-PB.pdf>.
- [27] Haochen Sun, Jason Li, and Hongyang Zhang. zkLLM: Zero knowledge proofs for large language models. In *ACM SIGSAC Conference on Computer and Communications Security*, pages 4405–4419. Association for Computing Machinery, 2024. doi: 10.1145/3658644.3670334. URL [https://hongyanz.github.io/publications/CCS\\_zkLLM.pdf](https://hongyanz.github.io/publications/CCS_zkLLM.pdf).
- [28] Jason Teutsch and Christian Reitwießner. A scalable verification solution for blockchains. arXiv:1908.04756, 2019. URL <https://arxiv.org/abs/1908.04756>. Earlier technical report dated November 16, 2017.
- [29] Zhaohui Wang. NanoZK: Privacy-preserving verifiable inference for large language models via layerwise zero-knowledge proofs. In *Information and Communications Security—ICICS 2026*. Springer, 2026. URL <https://arxiv.org/abs/2603.18046v2>. To appear; extended version arXiv:2603.18046v2, July 18, 2026.
- [30] Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger, 2025. URL <https://ethereum.github.io/yellowpaper/paper.pdf>. Yellow Paper, Shanghai version efc5f9a, February 4, 2025; accessed September 20, 2026.
- [31] Tiancheng Xie, Tao Lu, Zhiyong Fang, Siqi Wang, Zhenfei Zhang, Yongzheng Jia, Dawn Song, and Jiaheng Zhang. zkPyTorch: A hierarchical optimized compiler for zero-knowledge machine learning. Cryptology ePrint Archive, Paper 2025/535, 2025. URL <https://eprint.iacr.org/2025/535>.